

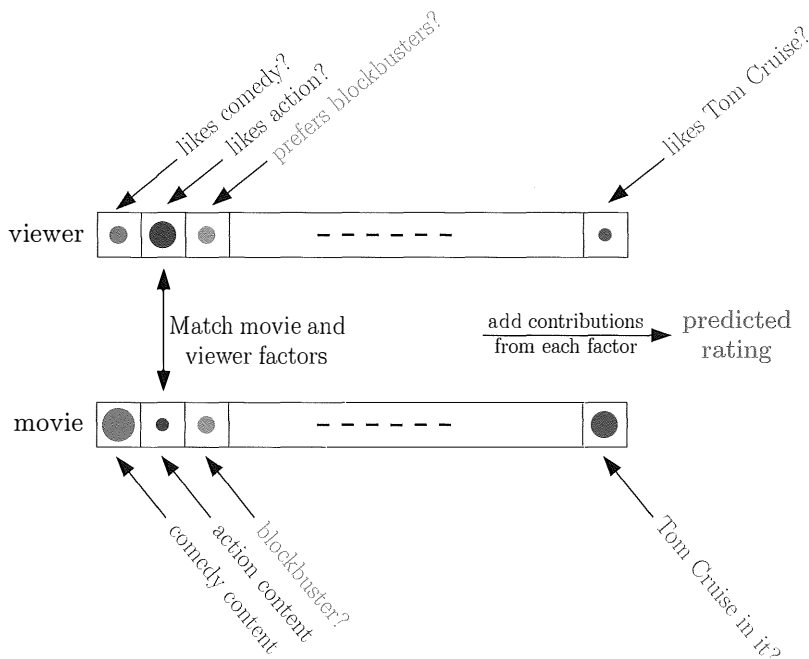


Figure 1.1: A model for how a viewer rates a movie

know that the historical rating data reveal a lot about how people rate movies, so we may be able to construct a good empirical solution. There is a great deal of data available to movie rental companies, since they often ask their viewers to rate the movies that they have already seen.

Figure 1.1 illustrates a specific approach that was widely used in the million-dollar competition. Here is how it works. You describe a movie as a long array of different factors, e.g., how much comedy is in it, how complicated is the plot, how handsome is the lead actor, etc. Now, you describe each viewer with corresponding factors; how much do they like comedy, do they prefer simple or complicated plots, how important are the looks of the lead actor, and so on. How this viewer will rate that movie is now estimated based on the match/mismatch of these factors. For example, if the movie is pure comedy and the viewer hates comedies, the chances are he won't like it. If you take dozens of these factors describing many facets of a movie's content and a viewer's taste, the conclusion based on matching all the factors will be a good predictor of how the viewer will rate the movie.

The power of learning from data is that this entire process can be automated, without any need for analyzing movie content or viewer taste. To do so, the learning algorithm 'reverse-engineers' these factors based solely on pre-

vious ratings. It starts with random factors, then tunes these factors to make them more and more aligned with how viewers have rated movies before, until they are ultimately able to *predict* how viewers rate movies in general. The factors we end up with may not be as intuitive as ‘comedy content’, and in fact can be quite subtle or even incomprehensible. After all, the algorithm is only trying to find the best way to predict how a viewer would rate a movie, not necessarily explain to us how it is done. This algorithm was part of the winning solution in the million-dollar competition.

1.1.1 Components of Learning

The movie rating application captures the essence of learning from data, and so do many other applications from vastly different fields. In order to abstract the common core of the learning problem, we will pick one application and use it as a metaphor for the different components of the problem. Let us take credit approval as our metaphor.

Suppose that a bank receives thousands of credit card applications every day, and it wants to automate the process of evaluating them. Just as in the case of movie ratings, the bank knows of no magical formula that can pinpoint when credit should be approved, but it has a lot of data. This calls for learning from data, so the bank uses historical records of previous customers to figure out a good formula for credit approval.

Each customer record has personal information related to credit, such as annual salary, years in residence, outstanding loans, etc. The record also keeps track of whether approving credit for that customer was a good idea, i.e., did the bank make money on that customer. This data guides the construction of a successful formula for credit approval that can be used on future applicants.

Let us give names and symbols to the main components of this learning problem. There is the input $\mathbf{x}$ (customer information that is used to make a credit decision), the unknown target function $f: \mathcal{X} \rightarrow \mathcal{Y}$ (ideal formula for credit approval), where $\mathcal{X}$ is the input space (set of all possible inputs $\mathbf{x}$), and $\mathcal{Y}$ is the output space (set of all possible outputs, in this case just a yes/no decision). There is a data set $\mathcal{D}$ of input-output examples $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$, where $y_n = f(\mathbf{x}_n)$ for $n = 1, \dots, N$ (inputs corresponding to previous customers and the correct credit decision for them in hindsight). The examples are often referred to as data points. Finally, there is the learning algorithm that uses the data set $\mathcal{D}$ to pick a formula $g: \mathcal{X} \rightarrow \mathcal{Y}$ that approximates f . The algorithm chooses g from a set of candidate formulas under consideration, which we call the hypothesis set $\mathcal{H}$. For instance, $\mathcal{H}$ could be the set of all linear formulas from which the algorithm would choose the best linear fit to the data, as we will introduce later in this section.

When a new customer applies for credit, the bank will base its decision on g (the hypothesis that the learning algorithm produced), not on f (the ideal target function which remains unknown). The decision will be good only to the extent that g faithfully replicates f . To achieve that, the algorithm

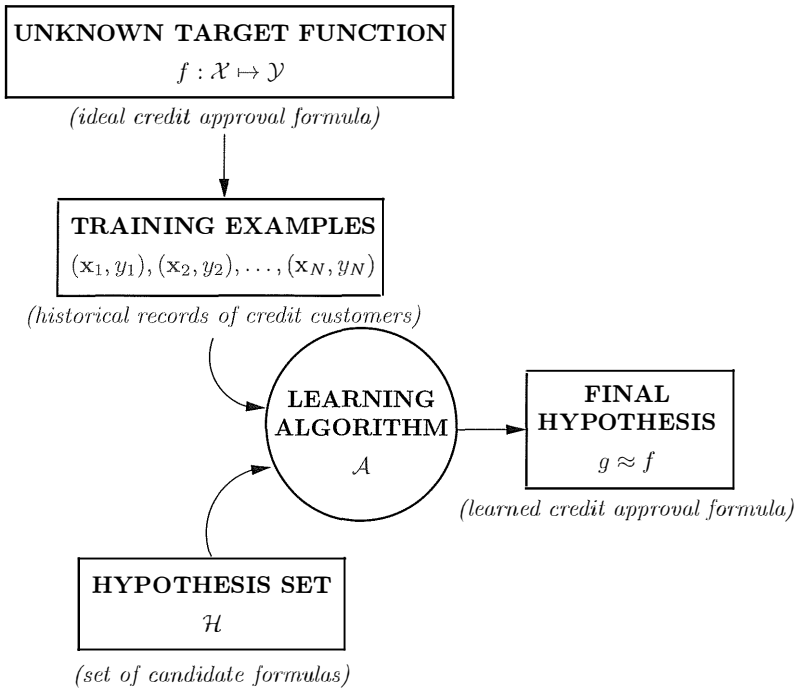


Figure 1.2: Basic setup of the learning problem

chooses g that best matches f on the *training* examples of previous customers, with the hope that it will continue to match f on new customers. Whether or not this hope is justified remains to be seen. Figure 1.2 illustrates the components of the learning problem.

Exercise 1.1

Express each of the following tasks in the framework of learning from data by specifying the input space $\mathcal{X}$, output space $\mathcal{Y}$, target function $f: \mathcal{X} \rightarrow \mathcal{Y}$, and the specifics of the data set that we will learn from.

- Medical diagnosis: A patient walks in with a medical history and some symptoms, and you want to identify the problem.
- Handwritten digit recognition (for example postal zip code recognition for mail sorting).
- Determining if an email is spam or not.
- Predicting how an electric load varies with price, temperature, and day of the week.
- A problem of interest to you for which there is no analytic solution, but you have data from which to construct an empirical solution.

We will use the setup in Figure 1.2 as our definition of the learning problem. Later on, we will consider a number of refinements and variations to this basic setup as needed. However, the essence of the problem will remain the same. There is a target to be learned. It is unknown to us. We have a set of examples generated by the target. The learning algorithm uses these examples to look for a hypothesis that approximates the target.

1.1.2 A Simple Learning Model

Let us consider the different components of Figure 1.2. Given a specific learning problem, the target function and training examples are dictated by the problem. However, the learning algorithm and hypothesis set are not. These are solution tools that we get to choose. The hypothesis set and learning algorithm are referred to informally as the *learning model*.

Here is a simple model. Let $\mathcal{X} = \mathbb{R}^d$ be the input space, where $\mathbb{R}^d$ is the d -dimensional Euclidean space, and let $\mathcal{Y} = \{+1, -1\}$ be the output space, denoting a binary (yes/no) decision. In our credit example, different coordinates of the input vector $\mathbf{x} \in \mathbb{R}^d$ correspond to salary, years in residence, outstanding debt, and the other data fields in a credit application. The binary output y corresponds to approving or denying credit. We specify the hypothesis set $\mathcal{H}$ through a functional form that all the hypotheses $h \in \mathcal{H}$ share. The functional form $h(\mathbf{x})$ that we choose here gives different weights to the different coordinates of $\mathbf{x}$, reflecting their relative importance in the credit decision. The weighted coordinates are then combined to form a ‘credit score’ and the result is compared to a threshold value. If the applicant passes the threshold, credit is approved; if not, credit is denied:

$$\begin{aligned} \text{Approve credit if } & \sum_{i=1}^d w_i x_i > \text{threshold}, \\ \text{Deny credit if } & \sum_{i=1}^d w_i x_i < \text{threshold}. \end{aligned}$$

This formula can be written more compactly as

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) + b \right), \quad (1.1)$$

where $x_1, \dots, x_d$ are the components of the vector $\mathbf{x}$; $h(\mathbf{x}) = +1$ means ‘approve credit’ and $h(\mathbf{x}) = -1$ means ‘deny credit’; $\text{sign}(s) = +1$ if $s > 0$ and $\text{sign}(s) = -1$ if $s < 0$.¹ The weights are $w_1, \dots, w_d$, and the threshold is determined by the bias term b since in Equation (1.1), credit is approved if $\sum_{i=1}^d w_i x_i > -b$.

This model of $\mathcal{H}$ is called the *perceptron*, a name that it got in the context of artificial intelligence. The learning algorithm will search $\mathcal{H}$ by looking for

¹The value of $\text{sign}(s)$ when $s = 0$ is a simple technicality that we ignore for the moment.

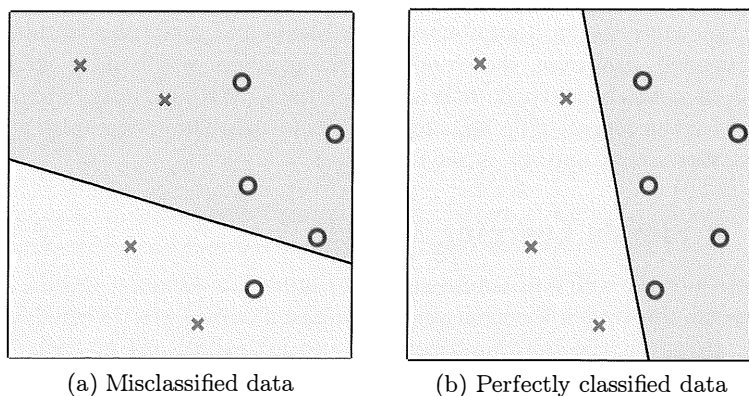


Figure 1.3: Perceptron classification of linearly separable data in a two dimensional input space (a) Some training examples will be misclassified (blue points in red region and vice versa) for certain values of the weight parameters which define the separating line. (b) A final hypothesis that classifies all training examples correctly. (o is +1 and $\times$ is -1.)

weights and bias that perform well on the data set. Some of the weights $w_1, \dots, w_d$ may end up being negative, corresponding to an adverse effect on credit approval. For instance, the weight of the ‘outstanding debt’ field should come out negative since more debt is not good for credit. The bias value b may end up being large or small, reflecting how lenient or stringent the bank should be in extending credit. The optimal choices of weights and bias define the final hypothesis $g \in \mathcal{H}$ that the algorithm produces.

Exercise 1.2

Suppose that we use a perceptron to detect spam messages. Let’s say that each email message is represented by the frequency of occurrence of keywords, and the output is +1 if the message is considered spam.

- (a) Can you think of some keywords that will end up with a large positive weight in the perceptron?
- (b) How about keywords that will get a negative weight?
- (c) What parameter in the perceptron directly affects how many borderline messages end up being classified as spam?

Figure 1.3 illustrates what a perceptron does in a two-dimensional case ($d = 2$). The plane is split by a line into two regions, the +1 decision region and the -1 decision region. Different values for the parameters w_1, w_2, b correspond to different lines $w_1x_1 + w_2x_2 + b = 0$. If the data set is *linearly separable*, there will be a choice for these parameters that classifies all the training examples correctly.

To simplify the notation of the perceptron formula, we will treat the bias b as a weight $w_0 = b$ and merge it with the other weights into one vector $\mathbf{w} = [w_0, w_1, \dots, w_d]^T$, where T denotes the transpose of a vector, so $\mathbf{w}$ is a column vector. We also treat $\mathbf{x}$ as a column vector and modify it to become $\mathbf{x} = [x_0, x_1, \dots, x_d]^T$, where the added coordinate x_0 is fixed at $x_0 = 1$. Formally speaking, the input space is now

$$\mathcal{X} = \{1\} \times \mathbb{R}^d = \{[x_0, x_1, \dots, x_d]^T \mid x_0 = 1, x_1 \in \mathbb{R}, \dots, x_d \in \mathbb{R}\}.$$

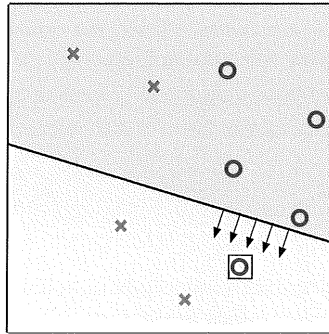
With this convention, $\mathbf{w}^T \mathbf{x} = \sum_{i=0}^d w_i x_i$, and so Equation (1.1) can be rewritten in vector form as

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x}). \quad (1.2)$$

We now introduce the *perceptron learning algorithm* (PLA). The algorithm will determine what $\mathbf{w}$ should be, based on the data. Let us assume that the data set is linearly separable, which means that there is a vector $\mathbf{w}$ that makes (1.2) achieve the correct decision $h(\mathbf{x}_n) = y_n$ on all the training examples, as shown in Figure 1.3.

Our learning algorithm will find this $\mathbf{w}$ using a simple iterative method. Here is how it works. At iteration t , where $t = 0, 1, 2, \dots$, there is a current value of the weight vector, call it $\mathbf{w}(t)$. The algorithm picks an example from $(\mathbf{x}_1, y_1) \dots (\mathbf{x}_N, y_N)$ that is currently misclassified, call it $(\mathbf{x}(t), y(t))$, and uses it to update $\mathbf{w}(t)$. Since the example is misclassified, we have $y(t) \neq \text{sign}(\mathbf{w}^T(t)\mathbf{x}(t))$. The update rule is

$$\mathbf{w}(t+1) = \mathbf{w}(t) + y(t)\mathbf{x}(t). \quad (1.3)$$



This rule moves the boundary in the direction of classifying $\mathbf{x}(t)$ correctly, as depicted in the figure above. The algorithm continues with further iterations until there are no longer misclassified examples in the data set.

Exercise 1.3

The weight update rule in (1.3) has the nice interpretation that it moves in the direction of classifying $\mathbf{x}(t)$ correctly.

- (a) Show that $y(t)\mathbf{w}^T(t)\mathbf{x}(t) < 0$. [*Hint: $\mathbf{x}(t)$ is misclassified by $\mathbf{w}(t)$.*]
- (b) Show that $y(t)\mathbf{w}^T(t+1)\mathbf{x}(t) > y(t)\mathbf{w}^T(t)\mathbf{x}(t)$. [*Hint: Use (1.3).*]
- (c) As far as classifying $\mathbf{x}(t)$ is concerned, argue that the move from $\mathbf{w}(t)$ to $\mathbf{w}(t+1)$ is a move ‘in the right direction’.

Although the update rule in (1.3) considers only one training example at a time and may ‘mess up’ the classification of the other examples that are not involved in the current iteration, it turns out that the algorithm is guaranteed to arrive at the right solution in the end. The proof is the subject of Problem 1.3. The result holds regardless of which example we choose from among the misclassified examples in $(\mathbf{x}_1, y_1) \cdots (\mathbf{x}_N, y_N)$ at each iteration, and regardless of how we initialize the weight vector to start the algorithm. For simplicity, we can pick one of the misclassified examples at random (or cycle through the examples and always choose the first misclassified one), and we can initialize $\mathbf{w}(0)$ to the zero vector.

Within the infinite space of all weight vectors, the perceptron algorithm manages to find a weight vector that works, using a simple iterative process. This illustrates how a learning algorithm can effectively search an infinite hypothesis set using a finite number of simple steps. This feature is characteristic of many techniques that are used in learning, some of which are far more sophisticated than the perceptron learning algorithm.

Exercise 1.4

Let us create our own target function f and data set $\mathcal{D}$ and see how the perceptron learning algorithm works. Take $d = 2$ so you can visualize the problem, and choose a random line in the plane as your target function, where one side of the line maps to $+1$ and the other maps to -1 . Choose the inputs $\mathbf{x}_n$ of the data set as random points in the plane, and evaluate the target function on each $\mathbf{x}_n$ to get the corresponding output y_n .

Now, generate a data set of size 20. Try the perceptron learning algorithm on your data set and see how long it takes to converge and how well the final hypothesis g matches your target f . You can find other ways to play with this experiment in Problem 1.4.

The perceptron learning algorithm succeeds in achieving its goal; finding a hypothesis that classifies all the points in the data set $\mathcal{D} = \{(\mathbf{x}_1, y_1) \cdots (\mathbf{x}_N, y_N)\}$ correctly. Does this mean that this hypothesis will also be successful in classifying new data points that are not in $\mathcal{D}$? This turns out to be the key question in the theory of learning, a question that will be thoroughly examined in this book.

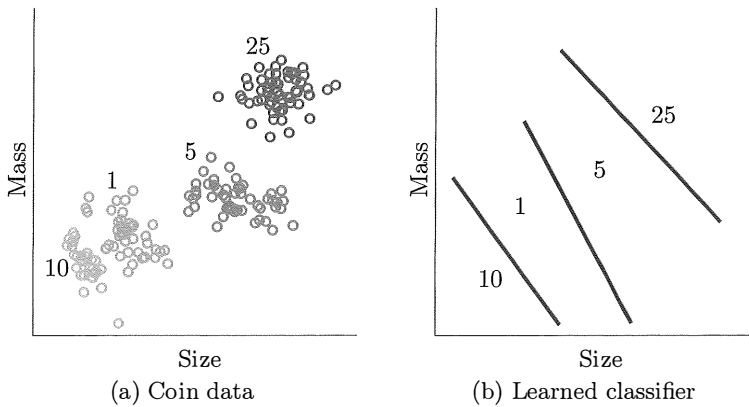


Figure 1.4: The learning approach to coin classification (a) Training data of pennies, nickels, dimes, and quarters (1, 5, 10, and 25 cents) are represented in a size mass space where they fall into clusters. (b) A classification rule is learned from the data set by separating the four clusters. A new coin will be classified according to the region in the size mass plane that it falls into.

1.1.3 Learning versus Design

So far, we have discussed what learning is. Now, we discuss what it is not. The goal is to distinguish between learning and a related approach that is used for similar problems. While learning is based on data, this other approach does not use data. It is a ‘design’ approach based on specifications, and is often discussed alongside the learning approach in pattern recognition literature.

Consider the problem of recognizing coins of different denominations, which is relevant to vending machines, for example. We want the machine to recognize quarters, dimes, nickels and pennies. We will contrast the ‘learning from data’ approach and the ‘design from specifications’ approach for this problem. We assume that each coin will be represented by its size and mass, a two-dimensional input.

In the learning approach, we are given a sample of coins from each of the four denominations and we use these coins as our data set. We treat the size and mass as the input vector, and the denomination as the output. Figure 1.4(a) shows what the data set may look like in the input space. There is some variation of size and mass within each class, but by and large coins of the same denomination cluster together. The learning algorithm searches for a hypothesis that classifies the data set well. If we want to classify a new coin, the machine measures its size and mass, and then classifies it according to the learned hypothesis in Figure 1.4(b).

In the design approach, we call the United States Mint and ask them about the specifications of different coins. We also ask them about the number

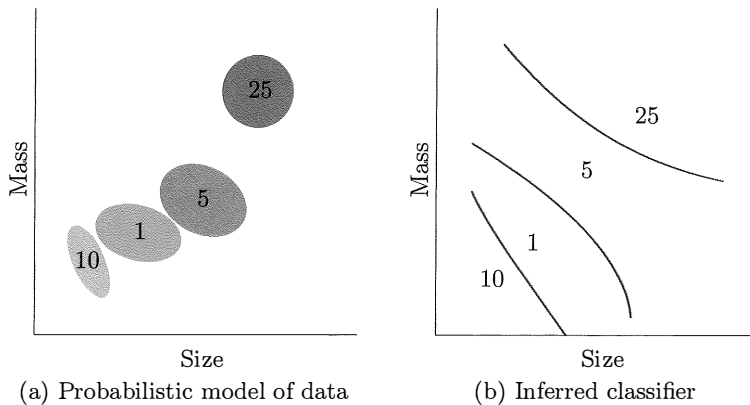


Figure 1.5: The design approach to coin classification (a) A probabilistic model for the size, mass, and denomination of coins is derived from known specifications. The figure shows the high probability region for each denomination (1, 5, 10, and 25 cents) according to the model. (b) A classification rule is derived analytically to minimize the probability of error in classifying a coin based on size and mass. The resulting regions for each denomination are shown.

of coins of each denomination in circulation, in order to get an estimate of the relative frequency of each coin. Finally, we make a physical model of the variations in size and mass due to exposure to the elements and due to errors in measurement. We put all of this information together and compute the full joint probability distribution of size, mass, and coin denomination (Figure 1.5(a)). Once we have that joint distribution, we can construct the optimal decision rule to classify coins based on size and mass (Figure 1.5(b)). The rule chooses the denomination that has the highest probability for a given size and mass, thus achieving the smallest possible probability of error.²

The main difference between the learning approach and the design approach is the role that data plays. In the design approach, the problem is well specified and one can analytically derive f without the need to see any data. In the learning approach, the problem is much less specified, and one needs data to pin down what f is.

Both approaches may be viable in some applications, but only the learning approach is possible in many applications where the target function is unknown. We are not trying to compare the utility or the performance of the two approaches. We are just making the point that the design approach is distinct from learning. This book is about learning.

²This is called Bayes optimal decision theory. Some learning models are based on the same theory by estimating the probability from data.

Exercise 1.5

Which of the following problems are more suited for the learning approach and which are more suited for the design approach?

- (a) Determining the age at which a particular medical test should be performed
- (b) Classifying numbers into primes and non-primes
- (c) Detecting potential fraud in credit card charges
- (d) Determining the time it would take a falling object to hit the ground
- (e) Determining the optimal cycle for traffic lights in a busy intersection

1.2 Types of Learning

The basic premise of learning from data is the use of a set of observations to uncover an underlying process. It is a very broad premise, and difficult to fit into a single framework. As a result, different learning paradigms have arisen to deal with different situations and different assumptions. In this section, we introduce some of these paradigms.

The learning paradigm that we have discussed so far is called *supervised* learning. It is the most studied and most utilized type of learning, but it is not the only one. Some variations of supervised learning are simple enough to be accommodated within the same framework. Other variations are more profound and lead to new concepts and techniques that take on lives of their own. The most important variations have to do with the nature of the data set.

1.2.1 Supervised Learning

When the training data contains explicit examples of what the correct output should be for given inputs, then we are within the supervised learning setting that we have covered so far. Consider the hand-written digit recognition problem (task (b) of Exercise 1.1). A reasonable data set for this problem is a collection of images of hand-written digits, and for each image, what the digit actually is. We thus have a set of examples of the form (image , digit). The learning is supervised in the sense that some ‘supervisor’ has taken the trouble to look at each input, in this case an image, and determine the correct output, in this case one of the ten categories $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

While we are on the subject of variations, there is more than one way that a data set can be presented to the learning process. Data sets are typically created and presented to us in their entirety at the outset of the learning process. For instance, historical records of customers in the credit-card application, and previous movie ratings of customers in the movie rating application, are already there for us to use. This protocol of a ‘ready’ data set is the most

common in practice, and it is what we will focus on in this book. However, it is worth noting that two variations of this protocol have attracted a significant body of work.

One is *active learning*, where the data set is acquired through queries that we make. Thus, we get to choose a point $\mathbf{x}$ in the input space, and the supervisor reports to us the target value for $\mathbf{x}$. As you can see, this opens the possibility for strategic choice of the point $\mathbf{x}$ to maximize its information value, similar to asking a strategic question in a game of 20 questions.

Another variation is called *online learning*, where the data set is given to the algorithm one example at a time. This happens when we have streaming data that the algorithm has to process ‘on the run’. For instance, when the movie recommendation system discussed in Section 1.1 is deployed, on-line learning can process new ratings from current users and movies. Online learning is also useful when we have limitations on computing and storage that preclude us from processing the whole data as a batch. We should note that online learning can be used in different paradigms of learning, not just in supervised learning.

1.2.2 Reinforcement Learning

When the training data does not explicitly contain the correct output for each input, we are no longer in a supervised learning setting. Consider a toddler learning not to touch a hot cup of tea. The experience of such a toddler would typically comprise a set of occasions when the toddler confronted a hot cup of tea and was faced with the decision of touching it or not touching it. Presumably, every time she touched it, the result was a high level of pain, and every time she didn’t touch it, a much lower level of pain resulted (that of an unsatisfied curiosity). Eventually, the toddler learns that she is better off not touching the hot cup.

The training examples did not spell out what the toddler should have done, but they instead graded different actions that she has taken. Nevertheless, she uses the examples to reinforce the better actions, eventually learning what she should do in similar situations. This characterizes *reinforcement* learning, where the training example does not contain the target output, but instead contains some possible output together with a measure of how good that output is. In contrast to supervised learning where the training examples were of the form (input , correct output), the examples in reinforcement learning are of the form

(input , some output , grade for this output).

Importantly, the example does not say how good other outputs would have been for this particular input.

Reinforcement learning is especially useful for learning how to play a game. Imagine a situation in backgammon where you have a choice between different actions and you want to identify the best action. It is not a trivial task to ascertain what the best action is at a given stage of the game, so we cannot

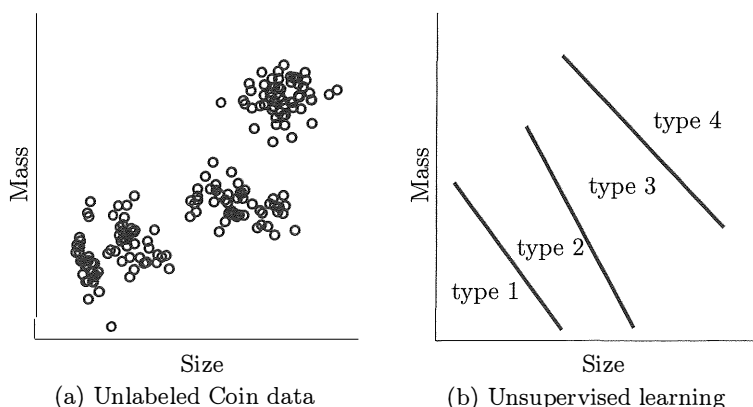


Figure 1.6: Unsupervised learning of coin classification (a) The same data set of coins in Figure 1.4(a) is again represented in the size mass space, but without being labeled. They still fall into clusters. (b) An unsupervised classification rule treats the four clusters as different types. The rule may be somewhat ambiguous, as type 1 and type 2 could be viewed as one cluster

easily create supervised learning examples. If you use reinforcement learning instead, all you need to do is to take some action and report how well things went, and you have a training example. The reinforcement learning algorithm is left with the task of sorting out the information coming from different examples to find the best line of play.

1.2.3 Unsupervised Learning

In the unsupervised setting, the training data does not contain any output information at all. We are just given input examples $\mathbf{x}_1, \dots, \mathbf{x}_N$. You may wonder how we could possibly learn anything from mere inputs. Consider the coin classification problem that we discussed earlier in Figure 1.4. Suppose that we didn't know the denomination of any of the coins in the data set. This *unlabeled data* is shown in Figure 1.6(a). We still get similar clusters, but they are now unlabeled so all points have the same 'color'. The decision regions in unsupervised learning may be identical to those in supervised learning, but without the labels (Figure 1.6(b)). However, the correct clustering is less obvious now, and even the number of clusters may be ambiguous.

Nonetheless, this example shows that we can learn *something* from the inputs by themselves. Unsupervised learning can be viewed as the task of spontaneously finding patterns and structure in input data. For instance, if our task is to categorize a set of books into topics, and we only use general properties of the various books, we can identify books that have similar properties and put them together in one category, without naming that category.

Unsupervised learning can also be viewed as a way to create a higher-level representation of the data. Imagine that you don't speak a word of Spanish, but your company will relocate you to Spain next month. They will arrange for Spanish lessons once you are there, but you would like to prepare yourself a bit before you go. All you have access to is a Spanish radio station. For a full month, you continuously bombard yourself with Spanish; this is an unsupervised learning experience since you don't know the meaning of the words. However, you gradually develop a better representation of the language in your brain by becoming more tuned to its common sounds and structures. When you arrive in Spain, you will be in a better position to start your Spanish lessons. Indeed, unsupervised learning can be a precursor to supervised learning. In other cases, it is a stand-alone technique.

Exercise 1.6

For each of the following tasks, identify which type of learning is involved (supervised, reinforcement, or unsupervised) and the training data to be used. If a task can fit more than one type, explain how and describe the training data for each type.

- (a) Recommending a book to a user in an online bookstore
- (b) Playing tic tac toe
- (c) Categorizing movies into different types
- (d) Learning to play music
- (e) Credit limit: Deciding the maximum allowed debt for each bank customer

Our main focus in this book will be supervised learning, which is the most popular form of learning from data.

1.2.4 Other Views of Learning

The study of learning has evolved somewhat independently in a number of fields that started historically at different times and in different domains, and these fields have developed different emphases and even different jargons. As a result, learning from data is a diverse subject with many aliases in the scientific literature. The main field dedicated to the subject is called *machine learning*, a name that distinguishes it from human learning. We briefly mention two other important fields that approach learning from data in their own ways.

Statistics shares the basic premise of learning from data, namely the use of a set of observations to uncover an underlying process. In this case, the process is a probability distribution and the observations are samples from that distribution. Because statistics is a mathematical field, emphasis is given to situations where most of the questions can be answered with rigorous proofs. As a result, statistics focuses on somewhat idealized models and analyzes them in great detail. This is the main difference between the statistical approach

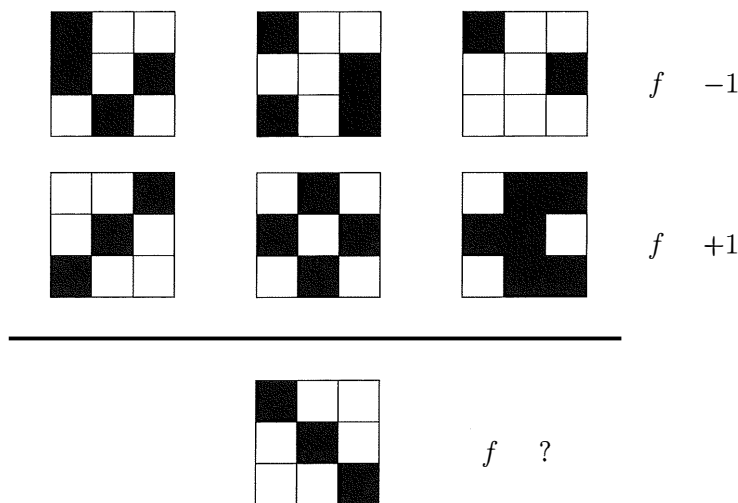


Figure 1.7: A visual learning problem. The first two rows show the training examples (each input $\mathbf{x}$ is a 9 bit vector represented visually as a 3×3 black and white array). The inputs in the first row have $f(\mathbf{x}) = -1$, and the inputs in the second row have $f(\mathbf{x}) = +1$. Your task is to learn from this data set what f is, then apply f to the test input at the bottom. Do you get -1 or $+1$?

to learning and how we approach the subject here. We make less restrictive assumptions and deal with more general models than in statistics. Therefore, we end up with weaker results that are nonetheless broadly applicable.

Data mining is a practical field that focuses on finding patterns, correlations, or anomalies in large relational databases. For example, we could be looking at medical records of patients and trying to detect a cause-effect relationship between a particular drug and long-term effects. We could also be looking at credit card spending patterns and trying to detect potential fraud. Technically, data mining is the same as learning from data, with more emphasis on data analysis than on prediction. Because databases are usually huge, computational issues are often critical in data mining. Recommender systems, which were illustrated in Section 1.1 with the movie rating example, are also considered part of data mining.

1.3 Is Learning Feasible?

The target function f is the object of learning. The most important assertion about the target function is that it is *unknown*. We really mean unknown.

This raises a natural question. How could a limited data set reveal enough information to pin down the entire target function? Figure 1.7 illustrates this

difficulty. A simple learning task with 6 training examples of a ± 1 target function is shown. Try to learn what the function is then apply it to the test input given. Do you get -1 or $+1$? Now, show the problem to your friends and see if they get the same answer.

The chances are the answers were not unanimous, and for good reason. There is simply more than one function that fits the 6 training examples, and some of these functions have a value of -1 on the test point and others have a value of $+1$. For instance, if the true f is $+1$ when the pattern is symmetric, the value for the test point would be $+1$. If the true f is $+1$ when the top left square of the pattern is white, the value for the test point would be -1 . Both functions agree with all the examples in the data set, so there isn't enough information to tell us which would be the correct answer.

This does not bode well for the feasibility of learning. To make matters worse, we will now see that the difficulty we experienced in this simple problem is the rule, not the exception.

1.3.1 Outside the Data Set

When we get the training data $\mathcal{D}$, e.g., the first two rows of Figure 1.7, we know the value of f on all the points in $\mathcal{D}$. This doesn't mean that we have learned f , since it doesn't guarantee that we know anything about f outside of $\mathcal{D}$. We know what we have already seen, but that's not learning. That's memorizing.

Does the data set $\mathcal{D}$ tell us anything outside of $\mathcal{D}$ that we didn't know before? If the answer is yes, then we have learned *something*. If the answer is no, we can conclude that learning is not feasible.

Since we maintain that f is an unknown function, we can prove that f remains unknown outside of $\mathcal{D}$. Instead of going through a formal proof for the general case, we will illustrate the idea in a concrete case. Consider a Boolean target function over a three-dimensional input space $\mathcal{X} = \{0, 1\}^3$. We are given a data set $\mathcal{D}$ of five examples represented in the table below. We denote the binary output by $\circ/\bullet$ for visual clarity,

$\mathbf{x}_n$	y_n
0 0 0	$\circ$
0 0 1	$\bullet$
0 1 0	$\bullet$
0 1 1	$\circ$
1 0 0	$\bullet$

where $y_n = f(\mathbf{x}_n)$ for $n = 1, 2, 3, 4, 5$. The advantage of this simple Boolean case is that we can enumerate the entire input space (since there are only $2^3 = 8$ distinct input vectors), and we can enumerate the set of all possible target functions (since f is a Boolean function on 3 Boolean inputs, and there are only $2^{2^3} = 256$ distinct Boolean functions on 3 Boolean inputs).

Let us look at the problem of learning f . Since f is unknown except inside $\mathcal{D}$, any function that agrees with $\mathcal{D}$ could conceivably be f . The table below shows all such functions $f_1, \dots, f_8$. It also shows the data set $\mathcal{D}$ (in blue) and what the final hypothesis g may look like.

$\mathbf{x}$	y	g	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8
0 0 0	○	○	○	○	○	○	○	○	○	○
0 0 1	●	●	●	●	●	●	●	●	●	●
0 1 0	●	●	●	●	●	●	●	●	●	●
0 1 1	○	○	○	○	○	○	○	○	○	○
1 0 0	●	●	●	●	●	●	●	●	●	●
1 0 1		?	○	○	○	○	●	●	●	●
1 1 0		?	○	○	●	●	○	○	●	●
1 1 1		?	○	●	○	●	○	●	○	●

The final hypothesis g is chosen based on the five examples in $\mathcal{D}$. The table shows the case where g is chosen to match f on these examples.

If we remain true to the notion of unknown target, we cannot exclude any of $f_1, \dots, f_8$ from being the true f . Now, we have a dilemma. The whole purpose of learning f is to be able to predict the value of f on points that we haven't seen before. The quality of the learning will be determined by how close our prediction is to the true value. Regardless of what g predicts on the three points we haven't seen before (those outside of $\mathcal{D}$, denoted by red question marks), it can agree or disagree with the target, depending on which of $f_1, \dots, f_8$ turns out to be the true target. It is easy to verify that any 3 bits that replace the red question marks are as good as any other 3 bits.

Exercise 1.7

For each of the following learning scenarios in the above problem, evaluate the performance of g on the three points in $\mathcal{X}$ outside $\mathcal{D}$. To measure the performance, compute how many of the 8 possible target functions agree with g on all three points, on two of them, on one of them, and on none of them.

- $\mathcal{H}$ has only two hypotheses, one that always returns '●' and one that always returns '○'. The learning algorithm picks the hypothesis that matches the data set the most.
- The same $\mathcal{H}$, but the learning algorithm now picks the hypothesis that matches the data set the *least*.
- $\mathcal{H} = \{\text{XOR}\}$ (only one hypothesis which is always picked), where XOR is defined by $\text{XOR}(\mathbf{x}) = \bullet$ if the number of 1's in $\mathbf{x}$ is odd and $\text{XOR}(\mathbf{x}) = \circ$ if the number is even.
- $\mathcal{H}$ contains all possible hypotheses (all Boolean functions on three variables), and the learning algorithm picks the hypothesis that agrees with all training examples, but otherwise disagrees the most with the XOR.

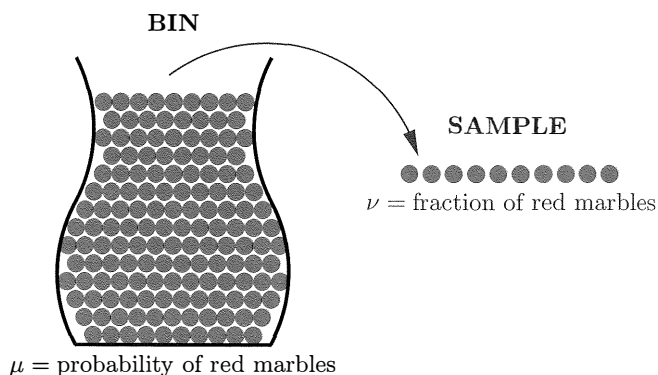


Figure 1.8: A random sample is picked from a bin of red and green marbles. The probability μ of red marbles in the bin is unknown. What does the fraction ν of red marbles in the sample tell us about μ ?

It doesn't matter what the algorithm does or what hypothesis set $\mathcal{H}$ is used. Whether $\mathcal{H}$ has a hypothesis that perfectly agrees with $\mathcal{D}$ (as depicted in the table) or not, and whether the learning algorithm picks that hypothesis or picks another one that disagrees with $\mathcal{D}$ (different green bits), it makes no difference whatsoever as far as the performance outside of $\mathcal{D}$ is concerned. Yet the performance outside $\mathcal{D}$ is all that matters in learning!

This dilemma is not restricted to Boolean functions, but extends to the general learning problem. As long as f is an unknown function, knowing $\mathcal{D}$ cannot exclude any pattern of values for f outside of $\mathcal{D}$. Therefore, the predictions of g outside of $\mathcal{D}$ are meaningless.

Does this mean that learning from data is doomed? If so, this will be a very short book 😊. Fortunately, learning is alive and well, and we will see why. We won't have to change our basic assumption to do that. The target function will continue to be unknown, and we still mean *unknown*.

1.3.2 Probability to the Rescue

We will show that we can indeed infer something outside $\mathcal{D}$ using only $\mathcal{D}$, but in a probabilistic way. What we infer may not be much compared to learning a full target function, but it will establish the principle that we can reach outside $\mathcal{D}$. Once we establish that, we will take it to the general learning problem and pin down what we can and cannot learn.

Let's take the simplest case of picking a sample, and see when we can say something about the objects outside the sample. Consider a bin that contains red and green marbles, possibly infinitely many. The proportion of red and green marbles in the bin is such that if we pick a marble at random, the probability that it will be red is μ and the probability that it will be green is $1 - \mu$. We assume that the value of μ is unknown to us.

We pick a random sample of N independent marbles (with replacement) from this bin, and observe the fraction ν of red marbles within the sample (Figure 1.8). What does the value of ν tell us about the value of μ ?

One answer is that regardless of the colors of the N marbles that we picked, we still don't know the color of any marble that we didn't pick. We can get mostly green marbles in the sample while the bin has mostly red marbles. Although this is certainly *possible*, it is by no means *probable*.

Exercise 1.8

If $\mu = 0.9$, what is the probability that a sample of 10 marbles will have $\nu \leq 0.1$? [Hints: 1. Use binomial distribution. 2. The answer is a very small number.]

The situation is similar to taking a poll. A random sample from a population tends to agree with the views of the population at large. The probability distribution of the random variable ν in terms of the parameter μ is well understood, and when the sample size is big, ν tends to be close to μ .

To quantify the relationship between ν and μ , we use a simple bound called the *Hoeffding Inequality*. It states that for any sample size N ,

$$\mathbb{P}[|\nu - \mu| > \epsilon] \leq 2e^{-2\epsilon^2 N} \quad \text{for any } \epsilon > 0. \quad (1.4)$$

Here, $\mathbb{P}[\cdot]$ denotes the probability of an event, in this case with respect to the random sample we pick, and ϵ is any positive value we choose. Putting Inequality (1.4) in words, it says that as the sample size N grows, it becomes exponentially unlikely that ν will deviate from μ by more than our 'tolerance' ϵ .

The only quantity that is random in (1.4) is ν which depends on the random sample. By contrast, μ is not random. It is just a constant, albeit unknown to us. There is a subtle point here. The utility of (1.4) is to infer the value of μ using the value of ν , although it is μ that affects ν , not vice versa. However, since the effect is that ν tends to be close to μ , we infer that μ 'tends' to be close to ν .

Although $\mathbb{P}[|\nu - \mu| > \epsilon]$ depends on μ , as μ appears in the argument and also affects the distribution of ν , we are able to bound the probability by $2e^{-2\epsilon^2 N}$ which does not depend on μ . Notice that only the size N of the sample affects the bound, not the size of the bin. The bin can be large or small, finite or infinite, and we still get the same bound when we use the same sample size.

Exercise 1.9

If $\mu = 0.9$, use the Hoeffding Inequality to bound the probability that a sample of 10 marbles will have $\nu \leq 0.1$ and compare the answer to the previous exercise.

If we choose ϵ to be very small in order to make ν a good approximation of μ , we need a larger sample size N to make the RHS of Inequality (1.4) small. We

can then assert that it is likely that ν will indeed be a good approximation of μ . Although this assertion does not give us the exact value of μ , and doesn't even guarantee that the approximate value holds, knowing that we are within $\pm\epsilon$ of μ most of the time is a significant improvement over not knowing anything at all.

The fact that the sample was randomly selected from the bin is the reason we are able to make any kind of statement about μ being close to ν . If the sample was not randomly selected but picked in a particular way, we would lose the benefit of the probabilistic analysis and we would again be in the dark outside of the sample.

How does the bin model relate to the learning problem? It seems that the unknown here was just the value of μ while the unknown in learning is an entire function $f: \mathcal{X} \rightarrow \mathcal{Y}$. The two situations can be connected. Take any single hypothesis $h \in \mathcal{H}$ and compare it to f on each point $\mathbf{x} \in \mathcal{X}$. If $h(\mathbf{x}) = f(\mathbf{x})$, color the point $\mathbf{x}$ green. If $h(\mathbf{x}) \neq f(\mathbf{x})$, color the point $\mathbf{x}$ red. The color that each point gets is not known to us, since f is unknown. However, if we pick $\mathbf{x}$ at random according to some probability distribution P over the input space $\mathcal{X}$, we know that $\mathbf{x}$ will be red with some probability, call it μ , and green with probability $1 - \mu$. Regardless of the value of μ , the space $\mathcal{X}$ now behaves like the bin in Figure 1.8.

The training examples play the role of a sample from the bin. If the inputs $\mathbf{x}_1, \dots, \mathbf{x}_N$ in $\mathcal{D}$ are picked independently according to P , we will get a random sample of red ($h(\mathbf{x}_n) \neq f(\mathbf{x}_n)$) and green ($h(\mathbf{x}_n) = f(\mathbf{x}_n)$) points. Each point will be red with probability μ and green with probability $1 - \mu$. The color of each point will be known to us since both $h(\mathbf{x}_n)$ and $f(\mathbf{x}_n)$ are known for $n = 1, \dots, N$ (the function h is our hypothesis so we can evaluate it on any point, and $f(\mathbf{x}_n) = y_n$ is given to us for all points in the data set $\mathcal{D}$). The learning problem is now reduced to a bin problem, under the assumption that the inputs in $\mathcal{D}$ are picked independently according to some distribution P on $\mathcal{X}$. Any P will translate to some μ in the equivalent bin. Since μ is allowed to be unknown, P can be unknown to us as well. Figure 1.9 adds this probabilistic component to the basic learning setup depicted in Figure 1.2.

With this equivalence, the Hoeffding Inequality can be applied to the learning problem, allowing us to make a prediction outside of $\mathcal{D}$. Using ν to predict μ tells us something about f , although it doesn't tell us what f is. What μ tells us is the error rate h makes in approximating f . If ν happens to be close to zero, we can predict that h will approximate f well over the entire input space. If not, we are out of luck.

Unfortunately, we have no control over ν in our current situation, since ν is based on a particular hypothesis h . In real learning, we explore an entire hypothesis set $\mathcal{H}$, looking for some $h \in \mathcal{H}$ that has a small error rate. If we have only one hypothesis to begin with, we are not really learning, but rather 'verifying' whether that particular hypothesis is good or bad. Let us see if we can extend the bin equivalence to the case where we have multiple hypotheses in order to capture real learning.

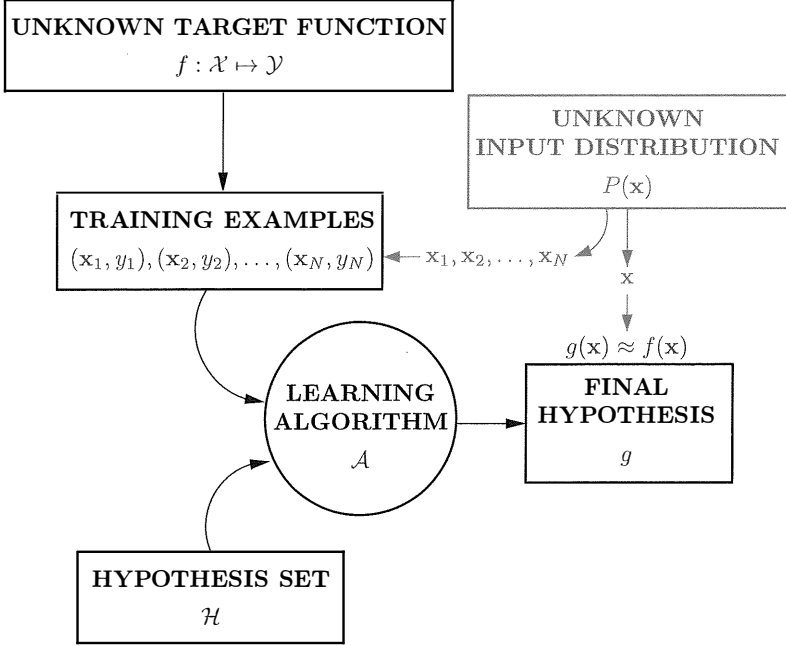


Figure 1.9: Probability added to the basic learning setup

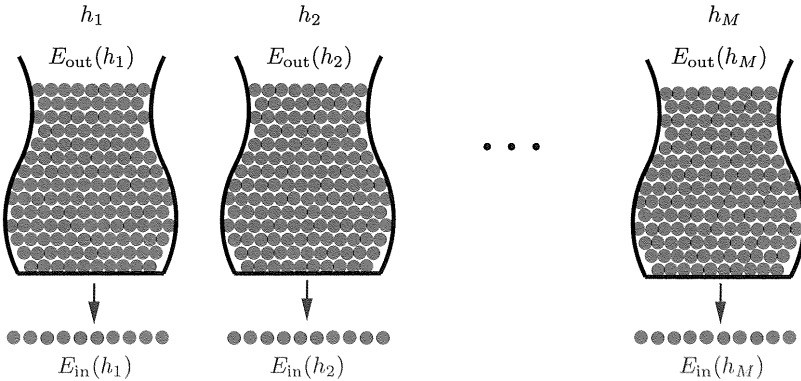
To do that, we start by introducing more descriptive names for the different components that we will use. The error rate within the sample, which corresponds to ν in the bin model, will be called the *in-sample error*,

$$\begin{aligned}
 E_{\text{in}}(h) &= (\text{fraction of } \mathcal{D} \text{ where } f \text{ and } h \text{ disagree}) \\
 &= \frac{1}{N} \sum_{n=1}^N \llbracket h(\mathbf{x}_n) \neq f(\mathbf{x}_n) \rrbracket,
 \end{aligned}$$

where $\llbracket \text{statement} \rrbracket = 1$ if the statement is true, and $= 0$ if the statement is false. We have made explicit the dependency of E_{in} on the particular h that we are considering. In the same way, we define the *out-of-sample error*

$$E_{\text{out}}(h) = \mathbb{P}[h(\mathbf{x}) \neq f(\mathbf{x})],$$

which corresponds to μ in the bin model. The probability is based on the distribution P over $\mathcal{X}$ which is used to sample the data points $\mathbf{x}$.

Figure 1.10: Multiple bins depict the learning problem with M hypotheses

Substituting the new notation E_{in} for ν and E_{out} for μ , the Hoeffding Inequality (1.4) can be rewritten as

$$\mathbb{P}[|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon] \leq 2e^{-2\epsilon^2 N} \quad \text{for any } \epsilon > 0, \quad (1.5)$$

where N is the number of training examples. The in-sample error E_{in} , just like ν , is a random variable that depends on the sample. The out-of-sample error E_{out} , just like μ , is unknown but not random.

Let us consider an entire hypothesis set $\mathcal{H}$ instead of just one hypothesis h , and assume for the moment that $\mathcal{H}$ has a finite number of hypotheses

$$\mathcal{H} = \{h_1, h_2, \dots, h_M\}.$$

We can construct a bin equivalent in this case by having M bins as shown in Figure 1.10. Each bin still represents the input space $\mathcal{X}$, with the red marbles in the m th bin corresponding to the points $\mathbf{x} \in \mathcal{X}$ where $h_m(\mathbf{x}) \neq f(\mathbf{x})$. The probability of red marbles in the m th bin is $E_{\text{out}}(h_m)$ and the fraction of red marbles in the m th sample is $E_{\text{in}}(h_m)$, for $m = 1, \dots, M$. Although the Hoeffding Inequality (1.5) still applies to each bin individually, the situation becomes more complicated when we consider all the bins simultaneously. Why is that? The inequality stated that

$$\mathbb{P}[|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon] \leq 2e^{-2\epsilon^2 N} \quad \text{for any } \epsilon > 0,$$

where the hypothesis h is *fixed* before you generate the data set, and the probability is with respect to random data sets $\mathcal{D}$; we emphasize that the assumption “ h is fixed *before* you generate the data set” is critical to the validity of this bound. If you are allowed to change h after you generate the data set, the assumptions that are needed to prove the Hoeffding Inequality no longer hold. With multiple hypotheses in $\mathcal{H}$, the learning algorithm picks

the final hypothesis g based on $\mathcal{D}$, i.e. *after* generating the data set. The statement we would like to make is not

$$“\mathbb{P}[|E_{\text{in}}(h_m) - E_{\text{out}}(h_m)| > \epsilon] \text{ is small}”$$

(for any particular, fixed $h_m \in \mathcal{H}$), but rather

$$“\mathbb{P}[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon] \text{ is small}” \text{ for the final hypothesis } g.$$

The hypothesis g is *not fixed* ahead of time before generating the data, because which hypothesis is selected to be g depends on the data. So, we cannot just plug in g for h in the Hoeffding inequality. The next exercise considers a simple coin experiment that further illustrates the difference between a fixed h and the final hypothesis g selected by the learning algorithm.

Exercise 1.10

Here is an experiment that illustrates the difference between a single bin and multiple bins. Run a computer simulation for flipping 1,000 fair coins. Flip each coin independently 10 times. Let's focus on 3 coins as follows: c_1 is the first coin flipped; c_{rand} is a coin you choose at random; c_{min} is the coin that had the minimum frequency of heads (pick the earlier one in case of a tie). Let ν_1 , ν_{rand} and ν_{min} be the fraction of heads you obtain for the respective three coins.

- What is μ for the three coins selected?
- Repeat this entire experiment a large number of times (e.g., 100,000 runs of the entire experiment) to get several instances of ν_1 , ν_{rand} and ν_{min} and plot the histograms of the distributions of ν_1 , ν_{rand} and ν_{min} . Notice that which coins end up being c_{rand} and c_{min} may differ from one run to another.
- Using (b), plot estimates for $\mathbb{P}[|\nu - \mu| > \epsilon]$ as a function of ϵ , together with the Hoeffding bound $2e^{-2\epsilon^2 N}$ (on the same graph).
- Which coins obey the Hoeffding bound, and which ones do not? Explain why.
- Relate part (d) to the multiple bins in Figure 1.10.

The way to get around this is to try to bound $\mathbb{P}[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon]$ in a way that does not depend on which g the learning algorithm picks. There is a simple but crude way of doing that. Since g has to be one of the h_m 's regardless of the algorithm and the sample, it is always true that

$$\begin{aligned} “|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon” &\implies “ \quad |E_{\text{in}}(h_1) - E_{\text{out}}(h_1)| > \epsilon \\ &\quad \text{or } |E_{\text{in}}(h_2) - E_{\text{out}}(h_2)| > \epsilon \\ &\quad \dots \\ &\quad \text{or } |E_{\text{in}}(h_M) - E_{\text{out}}(h_M)| > \epsilon ”. \end{aligned}$$

where $\mathcal{B}_1 \implies \mathcal{B}_2$ means that event $\mathcal{B}_1$ implies event $\mathcal{B}_2$. Although the events on the RHS cover a lot more than the LHS, the RHS has the property we want; the hypotheses h_m are fixed. We now apply two basic rules in probability;

$$\text{if } \mathcal{B}_1 \implies \mathcal{B}_2, \text{ then } \mathbb{P}[\mathcal{B}_1] \leq \mathbb{P}[\mathcal{B}_2],$$

and, if $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_M$ are any events, then

$$\mathbb{P}[\mathcal{B}_1 \text{ or } \mathcal{B}_2 \text{ or } \dots \text{ or } \mathcal{B}_M] \leq \mathbb{P}[\mathcal{B}_1] + \mathbb{P}[\mathcal{B}_2] + \dots + \mathbb{P}[\mathcal{B}_M].$$

The second rule is known as the *union bound*. Putting the two rules together, we get

$$\begin{aligned} \mathbb{P}[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon] &\leq \mathbb{P}[|E_{\text{in}}(h_1) - E_{\text{out}}(h_1)| > \epsilon \\ &\quad \text{or } |E_{\text{in}}(h_2) - E_{\text{out}}(h_2)| > \epsilon \\ &\quad \dots \\ &\quad \text{or } |E_{\text{in}}(h_M) - E_{\text{out}}(h_M)| > \epsilon] \\ &\leq \sum_{m=1}^M \mathbb{P}[|E_{\text{in}}(h_m) - E_{\text{out}}(h_m)| > \epsilon]. \end{aligned}$$

Applying the Hoeffding Inequality (1.5) to the M terms one at a time, we can bound each term in the sum by $2e^{-2\epsilon^2 N}$. Substituting, we get

$$\mathbb{P}[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon] \leq 2Me^{-2\epsilon^2 N}. \quad (1.6)$$

Mathematically, this is a ‘uniform’ version of (1.5). We are trying to simultaneously approximate all $E_{\text{out}}(h_m)$ ’s by the corresponding $E_{\text{in}}(h_m)$ ’s. This allows the learning algorithm to choose any hypothesis based on E_{in} and expect that the corresponding E_{out} will uniformly follow suit, regardless of which hypothesis is chosen.

The downside for uniform estimates is that the probability bound $2Me^{-2\epsilon^2 N}$ is a factor of M looser than the bound for a single hypothesis, and will only be meaningful if M is finite. We will improve on that in Chapter 2.

1.3.3 Feasibility of Learning

We have introduced two apparently conflicting arguments about the feasibility of learning. One argument says that we cannot learn anything outside of $\mathcal{D}$, and the other says that we can. We would like to reconcile these two arguments and pinpoint the sense in which learning is feasible:

1. Let us reconcile the two arguments. The question of whether $\mathcal{D}$ tells us anything outside of $\mathcal{D}$ that we didn’t know before has two different answers. If we insist on a deterministic answer, which means that $\mathcal{D}$ tells us something certain about f outside of $\mathcal{D}$, then the answer is no. If we accept a probabilistic answer, which means that $\mathcal{D}$ tells us something likely about f outside of $\mathcal{D}$, then the answer is yes.

Exercise 1.11

We are given a data set $\mathcal{D}$ of 25 training examples from an unknown target function $f: \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{X} = \mathbb{R}$ and $\mathcal{Y} = \{-1, +1\}$. To learn f , we use a simple hypothesis set $\mathcal{H} = \{h_1, h_2\}$ where h_1 is the constant $+1$ function and h_2 is the constant -1 .

We consider two learning algorithms, S (smart) and C (crazy). S chooses the hypothesis that agrees the most with $\mathcal{D}$ and C chooses the other hypothesis deliberately. Let us see how these algorithms perform out of sample from the deterministic and probabilistic points of view. Assume in the probabilistic view that there is a probability distribution on $\mathcal{X}$, and let $\mathbb{P}[f(x) = +1] = p$.

- (a) Can S produce a hypothesis that is *guaranteed* to perform better than random on any point outside $\mathcal{D}$?
- (b) Assume for the rest of the exercise that all the examples in $\mathcal{D}$ have $y_n = +1$. Is it *possible* that the hypothesis that C produces turns out to be better than the hypothesis that S produces?
- (c) If $p = 0.9$, what is the probability that S will produce a better hypothesis than C?
- (d) Is there any value of p for which it is more likely than not that C will produce a better hypothesis than S?

By adopting the probabilistic view, we get a positive answer to the feasibility question without paying too much of a price. The only assumption we make in the probabilistic framework is that the examples in $\mathcal{D}$ are generated independently. We don't insist on using any particular probability distribution, or even on knowing what distribution is used. However, whatever distribution we use for generating the examples, we must also use when we evaluate how well g approximates f (Figure 1.9). That's what makes the Hoeffding Inequality applicable. Of course this ideal situation may not always happen in practice, and some variations of it have been explored in the literature.

2. Let us pin down what we mean by the feasibility of learning. Learning produces a hypothesis g to approximate the unknown target function f . If learning is successful, then g should approximate f well, which means $E_{\text{out}}(g) \approx 0$. However, this is not what we get from the probabilistic analysis. What we get instead is $E_{\text{out}}(g) \approx E_{\text{in}}(g)$. We still have to make $E_{\text{in}}(g) \approx 0$ in order to conclude that $E_{\text{out}}(g) \approx 0$.

We cannot guarantee that we will find a hypothesis that achieves $E_{\text{in}}(g) \approx 0$, but at least we will know if we find it. Remember that $E_{\text{out}}(g)$ is an unknown quantity, since f is unknown, but $E_{\text{in}}(g)$ is a quantity that we can evaluate. We have thus traded the condition $E_{\text{out}}(g) \approx 0$, one that we cannot ascertain, for the condition $E_{\text{in}}(g) \approx 0$, which we can ascertain. What enabled this is the Hoeffding Inequality (1.6):

$$\mathbb{P}[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon] \leq 2Me^{-2\epsilon^2 N}$$

that assures us that $E_{\text{out}}(g) \approx E_{\text{in}}(g)$ so we can use E_{in} as a proxy for E_{out} .

Exercise 1.12

A friend comes to you with a learning problem. She says the target function f is *completely* unknown, but she has 4,000 data points. She is willing to pay you to solve her problem and produce for her a g which approximates f . What is the best that you can promise her among the following:

- (a) After learning you will provide her with a g that you will guarantee approximates f well out of sample.
- (b) After learning you will provide her with a g , and with high probability the g which you produce will approximate f well out of sample.
- (c) One of two things will happen.
 - (i) You will produce a hypothesis g ;
 - (ii) You will declare that you failed.

If you do return a hypothesis g , then with high probability the g which you produce will approximate f well out of sample.

One should note that there are cases where we won't insist that $E_{\text{in}}(g) \approx 0$. Financial forecasting is an example where market unpredictability makes it impossible to get a forecast that has anywhere near zero error. All we hope for is a forecast that gets it right more often than not. If we get that, our bets will win in the long run. This means that a hypothesis that has $E_{\text{in}}(g)$ somewhat below 0.5 will work, provided of course that $E_{\text{out}}(g)$ is close enough to $E_{\text{in}}(g)$.

The feasibility of learning is thus split into two questions:

1. Can we make sure that $E_{\text{out}}(g)$ is close enough to $E_{\text{in}}(g)$?
2. Can we make $E_{\text{in}}(g)$ small enough?

The Hoeffding Inequality (1.6) addresses the first question only. The second question is answered after we run the learning algorithm on the actual data and see how small we can get E_{in} to be.

Breaking down the feasibility of learning into these two questions provides further insight into the role that different components of the learning problem play. One such insight has to do with the 'complexity' of these components.

The complexity of $\mathcal{H}$. If the number of hypotheses M goes up, we run more risk that $E_{\text{in}}(g)$ will be a poor estimator of $E_{\text{out}}(g)$ according to Inequality (1.6). M can be thought of as a measure of the 'complexity' of the

hypothesis set $\mathcal{H}$ that we use. If we want an affirmative answer to the first question, we need to keep the complexity of $\mathcal{H}$ in check. However, if we want an affirmative answer to the second question, we stand a better chance if $\mathcal{H}$ is more complex, since g has to come from $\mathcal{H}$. So, a more complex $\mathcal{H}$ gives us more flexibility in finding some g that fits the data well, leading to small $E_{\text{in}}(g)$. This tradeoff in the complexity of $\mathcal{H}$ is a major theme in learning theory that we will study in detail in Chapter 2.

The complexity of f . Intuitively, a complex target function f should be harder to learn than a simple f . Let us examine if this can be inferred from the two questions above. A close look at Inequality (1.6) reveals that the complexity of f does not affect how well $E_{\text{in}}(g)$ approximates $E_{\text{out}}(g)$. If we fix the hypothesis set and the number of training examples, the inequality provides the same bound whether we are trying to learn a simple f (for instance a constant function) or a complex f (for instance a highly nonlinear function). However, this doesn't mean that we can learn complex functions as easily as we learn simple functions. Remember that (1.6) affects the first question only. If the target function is complex, the second question comes into play since the data from a complex f are harder to fit than the data from a simple f . This means that we will get a worse value for $E_{\text{in}}(g)$ when f is complex. We might try to get around that by making our hypothesis set more complex so that we can fit the data better and get a lower $E_{\text{in}}(g)$, but then E_{out} won't be as close to E_{in} per (1.6). Either way we look at it, a complex f is harder to learn as we expected. In the extreme case, if f is too complex, we may not be able to learn it at all.

Fortunately, most target functions in real life are not too complex; we can learn them from a reasonable $\mathcal{D}$ using a reasonable $\mathcal{H}$. This is obviously a practical observation, not a mathematical statement. Even when we cannot learn a particular f , we will at least be able to tell that we can't. As long as we make sure that the complexity of $\mathcal{H}$ gives us a good Hoeffding bound, our success or failure in learning f can be determined by our success or failure in fitting the training data.

1.4 Error and Noise

We close this chapter by revisiting two notions in the learning problem in order to bring them closer to the real world. The first notion is what approximation means when we say that our hypothesis approximates the target function well. The second notion is about the nature of the target function. In many situations, there is noise that makes the output of f not uniquely determined by the input. What are the ramifications of having such a 'noisy' target on the learning problem?

1.4.1 Error Measures

Learning is not expected to replicate the target function perfectly. The final hypothesis g is only an approximation of f . To quantify how well g approximates f , we need to define an error measure³ that quantifies how far we are from the target.

The choice of an error measure affects the outcome of the learning process. Different error measures may lead to different choices of the final hypothesis, even if the target and the data are the same, since the value of a particular error measure may be small while the value of another error measure in the same situation is large. Therefore, which error measure we use has consequences for what we learn. What are the criteria for choosing one error measure over another? We address this question here.

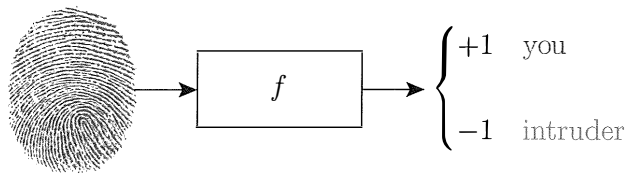
First, let's formalize this notion a bit. An error measure quantifies how well each hypothesis h in the model approximates the target function f ,

$$\text{Error} = E(h, f).$$

While $E(h, f)$ is based on the entirety of h and f , it is almost universally defined based on the errors on individual input points $\mathbf{x}$. If we define a pointwise error measure $e(h(\mathbf{x}), f(\mathbf{x}))$, the overall error will be the average value of this pointwise error. So far, we have been working with the classification error $e(h(\mathbf{x}), f(\mathbf{x})) = \llbracket h(\mathbf{x}) \neq f(\mathbf{x}) \rrbracket$.

In an ideal world, $E(h, f)$ should be user-specified. The same learning task in different contexts may warrant the use of different error measures. One may view $E(h, f)$ as the 'cost' of using h when you should use f . This cost depends on what h is used for, and cannot be dictated just by our learning techniques. Here is a case in point.

Example 1.1 (Fingerprint verification). Consider the problem of verifying that a fingerprint belongs to a particular person. What is the appropriate error measure?



The target function takes as input a fingerprint, and returns $+1$ if it belongs to the right person, and -1 if it belongs to an intruder.

³This measure is also called an error *function* in the literature, and sometimes the error is referred to as *cost*, *objective*, or *risk*.

There are two types of error that our hypothesis h can make here. If the correct person is rejected ($h = -1$ but $f = +1$), it is called false reject, and if an incorrect person is accepted ($h = +1$ but $f = -1$), it is called false accept.

		f	
		+1	-1
h	+1	no error	false accept
	-1	false reject	no error

How should the error measure be defined in this problem? If the right person is accepted or an intruder is rejected, the error is clearly zero. We need to specify the error values for a false accept and for a false reject. The right values depend on the application.

Consider two potential clients of this fingerprint system. One is a supermarket who will use it at the checkout counter to verify that you are a member of a discount program. The other is the CIA who will use it at the entrance to a secure facility to verify that you are authorized to enter that facility.

For the supermarket, a false reject is costly because if a customer gets wrongly rejected, she may be discouraged from patronizing the supermarket in the future. All future revenue from this annoyed customer is lost. On the other hand, the cost of a false accept is minor. You just gave away a discount to someone who didn't deserve it, and that person left their fingerprint in your system — they must be bold indeed.

For the CIA, a false accept is a disaster. An unauthorized person will gain access to a highly sensitive facility. This should be reflected in a much higher cost for the false accept. False rejects, on the other hand, can be tolerated since authorized persons are employees (rather than customers as with the supermarket). The inconvenience of retrying when rejected is just part of the job, and they must deal with it.

The costs of the different types of errors can be tabulated in a matrix. For our examples, the matrices might look like:

		f	
		+1	-1
h	+1	0	1
	-1	10	0

Supermarket

		f	
		+1	-1
h	+1	0	1000
	-1	1	0

CIA

These matrices should be used to weight the different types of errors when we compute the total error. When the learning algorithm minimizes a cost-weighted error measure, it automatically takes into consideration the utility of the hypothesis that it will produce. In the supermarket and CIA scenarios, this could lead to two completely different final hypotheses. $\square$

The moral of this example is that the choice of the error measure depends on how the system is going to be used, rather than on any inherent criterion

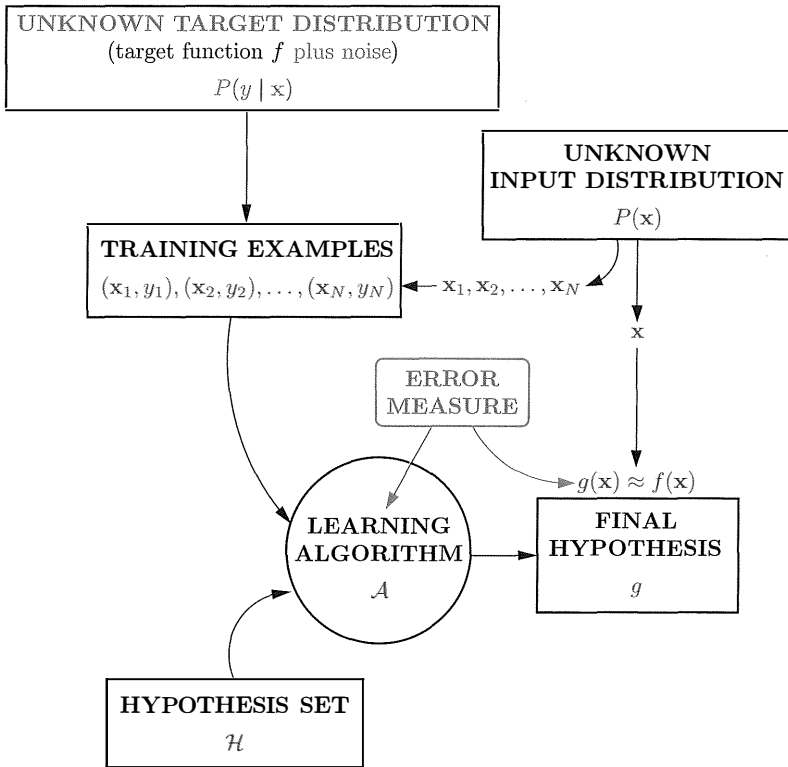


Figure 1.11: The general (supervised) learning problem

that we can independently determine during the learning process. However, this ideal choice may not be possible in practice for two reasons. One is that the user may not provide an error specification, which is not uncommon. The other is that the weighted cost may be a difficult objective function for optimizers to work with. Therefore, we often look for other ways to define the error measure, sometimes with purely practical or analytic considerations in mind. We have already seen an example of this with the simple binary error used in this chapter, and we will see other error measures in later chapters.

1.4.2 Noisy Targets

In many practical applications, the data we learn from are not generated by a deterministic target function. Instead, they are generated in a noisy way such that the output is not uniquely determined by the input. For instance, in the credit-card example we presented in Section 1.1, two customers may have identical salaries, outstanding loans, etc., but end up with different credit behavior. Therefore, the credit ‘function’ is not really a deterministic function,

but a noisy one.

This situation can be readily modeled within the same framework that we have. Instead of $y = f(\mathbf{x})$, we can take the output y to be a random variable that is affected by, rather than determined by, the input $\mathbf{x}$. Formally, we have a target *distribution* $P(y | \mathbf{x})$ instead of a target function $y = f(\mathbf{x})$. A data point $(\mathbf{x}, y)$ is now generated by the joint distribution $P(\mathbf{x}, y) = P(\mathbf{x})P(y | \mathbf{x})$.

One can think of a noisy target as a deterministic target plus added noise. If y is real-valued for example, one can take the expected value of y given $\mathbf{x}$ to be the deterministic $f(\mathbf{x})$, and consider $y - f(\mathbf{x})$ as pure noise that is added to f .

This view suggests that a deterministic target function can be considered a special case of a noisy target, just with zero noise. Indeed, we can formally express any function f as a distribution $P(y | \mathbf{x})$ by choosing $P(y | \mathbf{x})$ to be zero for all y except $y = f(\mathbf{x})$. Therefore, there is no loss of generality if we consider the target to be a distribution rather than a function. Figure 1.11 modifies the previous Figures 1.2 and 1.9 to illustrate the general learning problem, covering both deterministic and noisy targets.

Exercise 1.13

Consider the bin model for a hypothesis h that makes an error with probability μ in approximating a deterministic target function f (both h and f are binary functions). If we use the same h to approximate a noisy version of f given by

$$P(y | \mathbf{x}) = \begin{cases} \lambda & y = f(\mathbf{x}), \\ 1 - \lambda & y \neq f(\mathbf{x}). \end{cases}$$

- (a) What is the probability of error that h makes in approximating y ?
- (b) At what value of λ will the performance of h be independent of μ ?
[Hint: The noisy target will look completely random.]

There is a difference between the role of $P(y | \mathbf{x})$ and the role of $P(\mathbf{x})$ in the learning problem. While both distributions model probabilistic aspects of $\mathbf{x}$ and y , the target distribution $P(y | \mathbf{x})$ is what we are trying to learn, while the input distribution $P(\mathbf{x})$ only quantifies the relative importance of the point $\mathbf{x}$ in gauging how well we have learned.

Our entire analysis of the feasibility of learning applies to noisy target functions as well. Intuitively, this is because the Hoeffding Inequality (1.6) applies to an arbitrary, unknown target function. Assume we randomly picked all the y 's according to the distribution $P(y | \mathbf{x})$ over the entire input space $\mathcal{X}$. This realization of $P(y | \mathbf{x})$ is effectively a target function. Therefore, the inequality will be valid no matter which particular random realization the 'target function' happens to be.

This does not mean that learning a noisy target is as easy as learning a deterministic one. Remember the two questions of learning? With the same learning model, E_{out} may be as close to E_{in} in the noisy case as it is in the

deterministic case, but E_{in} itself will likely be worse in the noisy case since it is hard to fit the noise.

In Chapter 2, where we prove a stronger version of (1.6), we will assume the target to be a probability distribution $P(y \mid \mathbf{x})$, thus covering the general case.

1.5 Problems

Problem 1.1 We have 2 opaque bags, each containing 2 balls. One bag has 2 black balls and the other has a black and a white ball. You pick a bag at random and then pick one of the balls in that bag at random. When you look at the ball it is black. You now pick the second ball from that same bag. What is the probability that this ball is also black? [*Hint: Use Bayes' Theorem: $\mathbb{P}[A \text{ and } B] = \mathbb{P}[A | B] \mathbb{P}[B] = \mathbb{P}[B | A] \mathbb{P}[A].$*]

Problem 1.2 Consider the perceptron in two dimensions: $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$ where $\mathbf{w} = [w_0, w_1, w_2]^T$ and $\mathbf{x} = [1, x_1, x_2]^T$. Technically, $\mathbf{x}$ has three coordinates, but we call this perceptron two-dimensional because the first coordinate is fixed at 1.

- Show that the regions on the plane where $h(\mathbf{x}) = +1$ and $h(\mathbf{x}) = -1$ are separated by a line. If we express this line by the equation $x_2 = ax_1 + b$, what are the slope a and intercept b in terms of w_0, w_1, w_2 ?
- Draw a picture for the cases $\mathbf{w} = [1, 2, 3]^T$ and $\mathbf{w} = -[1, 2, 3]^T$.

In more than two dimensions, the $+1$ and -1 regions are separated by a *hyperplane*, the generalization of a line.

Problem 1.3 Prove that the PLA eventually converges to a linear separator for separable data. The following steps will guide you through the proof. Let $\mathbf{w}^*$ be an optimal set of weights (one which separates the data). The essential idea in this proof is to show that the PLA weights $\mathbf{w}(t)$ get "more aligned" with $\mathbf{w}^*$ with every iteration. For simplicity, assume that $\mathbf{w}(0) = \mathbf{0}$.

- Let $\rho = \min_{1 \leq n \leq N} y_n(\mathbf{w}^{*T} \mathbf{x}_n)$. Show that $\rho > 0$.
- Show that $\mathbf{w}^T(t) \mathbf{w}^* \geq \mathbf{w}^T(t-1) \mathbf{w}^* + \rho$, and conclude that $\mathbf{w}^T(t) \mathbf{w}^* \geq t\rho$. [*Hint: Use induction.*]
- Show that $\|\mathbf{w}(t)\|^2 \leq \|\mathbf{w}(t-1)\|^2 + \|\mathbf{x}(t-1)\|^2$.
[*Hint: $y(t-1) \cdot (\mathbf{w}^T(t-1) \mathbf{x}(t-1)) \leq 0$ because $\mathbf{x}(t-1)$ was misclassified by $\mathbf{w}(t-1)$.*]
- Show by induction that $\|\mathbf{w}(t)\|^2 \leq tR^2$, where $R = \max_{1 \leq n \leq N} \|\mathbf{x}_n\|$.

(continued on next page)

(e) Using (b) and (d), show that

$$\frac{\mathbf{w}^T(t) \mathbf{w}^*}{\|\mathbf{w}(t)\|} \geq \sqrt{t} \cdot \frac{\rho}{R},$$

and hence prove that

$$t \leq \frac{R^2 \|\mathbf{w}^*\|^2}{\rho^2}.$$

$$\left[\text{Hint: } \frac{\mathbf{w}^T(t) \mathbf{w}^*}{\|\mathbf{w}(t)\| \|\mathbf{w}^*\|} \leq 1. \text{ Why?} \right]$$

In practice, PLA converges more quickly than the bound $\frac{R^2 \|\mathbf{w}^*\|^2}{\rho^2}$ suggests. Nevertheless, because we do not know ρ in advance, we can't determine the number of iterations to convergence, which does pose a problem if the data is non-separable.

Problem 1.4 In Exercise 1.4, we use an artificial data set to study the perceptron learning algorithm. This problem leads you to explore the algorithm further with data sets of different sizes and dimensions.

- (a) Generate a linearly separable data set of size 20 as indicated in Exercise 1.4. Plot the examples $\{(\mathbf{x}_n, y_n)\}$ as well as the target function f on a plane. Be sure to mark the examples from different classes differently, and add labels to the axes of the plot.
- (b) Run the perceptron learning algorithm on the data set above. Report the number of updates that the algorithm takes before converging. Plot the examples $\{(\mathbf{x}_n, y_n)\}$, the target function f , and the final hypothesis g in the same figure. Comment on whether f is close to g .
- (c) Repeat everything in (b) with another randomly generated data set of size 20. Compare your results with (b).
- (d) Repeat everything in (b) with another randomly generated data set of size 100. Compare your results with (b).
- (e) Repeat everything in (b) with another randomly generated data set of size 1,000. Compare your results with (b).
- (f) Modify the algorithm such that it takes $\mathbf{x}_n \in \mathbb{R}^{10}$ instead of $\mathbb{R}^2$. Randomly generate a linearly separable data set of size 1,000 with $\mathbf{x}_n \in \mathbb{R}^{10}$ and feed the data set to the algorithm. How many updates does the algorithm take to converge?
- (g) Repeat the algorithm on the same data set as (f) for 100 experiments. In the iterations of each experiment, pick $\mathbf{x}(t)$ randomly instead of deterministically. Plot a histogram for the number of updates that the algorithm takes to converge.
- (h) Summarize your conclusions with respect to accuracy and running time as a function of N and d .

Problem 1.5 The perceptron learning algorithm works like this: In each iteration t , pick a random $(\mathbf{x}(t), y(t))$ and compute the 'signal' $s(t) = \mathbf{w}^T(t)\mathbf{x}(t)$. If $y(t) \cdot s(t) \leq 0$, update $\mathbf{w}$ by

$$\mathbf{w}(t+1) \leftarrow \mathbf{w}(t) + y(t) \cdot \mathbf{x}(t) ;$$

One may argue that this algorithm does not take the 'closeness' between $s(t)$ and $y(t)$ into consideration. Let's look at another perceptron learning algorithm: In each iteration, pick a random $(\mathbf{x}(t), y(t))$ and compute $s(t)$. If $y(t) \cdot s(t) \leq 1$, update $\mathbf{w}$ by

$$\mathbf{w}(t+1) \leftarrow \mathbf{w}(t) + \eta \cdot (y(t) - s(t)) \cdot \mathbf{x}(t) ,$$

where η is a constant. That is, if $s(t)$ agrees with $y(t)$ well (their product is > 1), the algorithm does nothing. On the other hand, if $s(t)$ is further from $y(t)$, the algorithm changes $\mathbf{w}(t)$ more. In this problem, you are asked to implement this algorithm and study its performance.

- (a) Generate a training data set of size 100 similar to that used in Exercise 1.4. Generate a test data set of size 10,000 from the same process. To get g , run the algorithm above with $\eta = 100$ on the training data set, until a maximum of 1,000 updates has been reached. Plot the training data set, the target function f , and the final hypothesis g on the same figure. Report the error on the test set.
- (b) Use the data set in (a) and redo everything with $\eta = 1$.
- (c) Use the data set in (a) and redo everything with $\eta = 0.01$.
- (d) Use the data set in (a) and redo everything with $\eta = 0.0001$.
- (e) Compare the results that you get from (a) to (d).

The algorithm above is a variant of the so called Adaline (*Adaptive Linear Neuron*) algorithm for perceptron learning.

Problem 1.6 Consider a sample of 10 marbles drawn independently from a bin that holds red and green marbles. The probability of a red marble is μ . For $\mu = 0.05$, $\mu = 0.5$, and $\mu = 0.8$, compute the probability of getting no red marbles ($\nu = 0$) in the following cases.

- (a) We draw only one such sample. Compute the probability that $\nu = 0$.
- (b) We draw 1,000 independent samples. Compute the probability that (at least) one of the samples has $\nu = 0$.
- (c) Repeat (b) for 1,000,000 independent samples.

Problem 1.7 A sample of heads and tails is created by tossing a coin a number of times independently. Assume we have a number of coins that generate different samples independently. For a given coin, let the probability of heads (probability of error) be μ . The probability of obtaining k heads in N tosses of this coin is given by the binomial distribution:

$$P[k | N, \mu] = \binom{N}{k} \mu^k (1 - \mu)^{N-k}.$$

Remember that the training error ν is $\frac{k}{N}$.

- (a) Assume the sample size (N) is 10. If all the coins have $\mu = 0.05$ compute the probability that at least one coin will have $\nu = 0$ for the case of 1 coin, 1,000 coins, 1,000,000 coins. Repeat for $\mu = 0.8$.
- (b) For the case $N = 6$ and 2 coins with $\mu = 0.5$ for both coins, plot the probability

$$P[\max_i |\nu_i - \mu_i| > \epsilon]$$

for ϵ in the range $[0, 1]$ (the max is over coins). On the same plot show the bound that would be obtained using the Hoeffding Inequality. Remember that for a single coin, the Hoeffding bound is

$$P[|\nu - \mu| > \epsilon] \leq 2e^{-2N\epsilon^2}.$$

[Hint: Use $P[A \text{ or } B] = P[A] + P[B]$ $P[A \text{ and } B] = P[A] + P[B] - P[A]P[B]$, where the last equality follows by independence, to evaluate $P[\max \dots]$]

Problem 1.8 The Hoeffding Inequality is one form of the *law of large numbers*. One of the simplest forms of that law is the *Chebyshev Inequality*, which you will prove here.

- (a) If t is a non negative random variable, prove that for any $\alpha > 0$, $\mathbb{P}[t \geq \alpha] \leq \mathbb{E}(t)/\alpha$.
- (b) If u is any random variable with mean μ and variance σ^2 , prove that for any $\alpha > 0$, $\mathbb{P}[(u - \mu)^2 \geq \alpha] \leq \frac{\sigma^2}{\alpha}$. [Hint: Use (a)]
- (c) If $u_1, \dots, u_N$ are iid random variables, each with mean μ and variance σ^2 , and $u = \frac{1}{N} \sum_{n=1}^N u_n$, prove that for any $\alpha > 0$,

$$\mathbb{P}[(u - \mu)^2 \geq \alpha] \leq \frac{\sigma^2}{N\alpha}.$$

Notice that the RHS of this Chebyshev Inequality goes down linearly in N , while the counterpart in Hoeffding's Inequality goes down exponentially. In Problem 1.9, we develop an exponential bound using a similar approach.

Problem 1.9 In this problem, we derive a form of the law of large numbers that has an exponential bound, called the *Chernoff bound*. We focus on the simple case of flipping a fair coin, and use an approach similar to Problem 1.8.

- (a) Let t be a (finite) random variable, α be a positive constant, and s be a positive parameter. If $T(s) = \mathbb{E}_t(e^{st})$, prove that

$$\mathbb{P}[t \geq \alpha] \leq e^{-s\alpha} T(s).$$

[Hint: e^{st} is monotonically increasing in t .]

- (b) Let $u_1, \dots, u_N$ be iid random variables, and let $u = \frac{1}{N} \sum_{n=1}^N u_n$. If $U(s) = \mathbb{E}_{u_n}(e^{su_n})$ (for any n), prove that

$$\mathbb{P}[u \geq \alpha] \leq (e^{-s\alpha} U(s))^N.$$

- (c) Suppose $\mathbb{P}[u_n = 0] = \mathbb{P}[u_n = 1] = \frac{1}{2}$ (fair coin). Evaluate $U(s)$ as a function of s , and minimize $e^{-s\alpha} U(s)$ with respect to s for fixed α , $0 < \alpha < 1$.
- (d) Conclude in (c) that, for $0 < \epsilon < \frac{1}{2}$,

$$\mathbb{P}[u \geq \mathbb{E}(u) + \epsilon] \leq 2^{-\beta N},$$

where $\beta = 1 + (\frac{1}{2} + \epsilon) \log_2(\frac{1}{2} + \epsilon) + (\frac{1}{2} - \epsilon) \log_2(\frac{1}{2} - \epsilon)$ and $\mathbb{E}(u) = \frac{1}{2}$. Show that $\beta > 0$, hence the bound is exponentially decreasing in N .

Problem 1.10 Assume that $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N, \mathbf{x}_{N+1}, \dots, \mathbf{x}_{N+M}\}$ and $\mathcal{Y} = \{-1, +1\}$ with an unknown target function $f: \mathcal{X} \rightarrow \mathcal{Y}$. The training data set $\mathcal{D}$ is $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$. Define the *off-training-set error* of a hypothesis h with respect to f by

$$E_{\text{off}}(h, f) = \frac{1}{M} \sum_{m=1}^M \mathbb{I}[h(\mathbf{x}_{N+m}) \neq f(\mathbf{x}_{N+m})].$$

- (a) Say $f(\mathbf{x}) = +1$ for all $\mathbf{x}$ and

$$h(\mathbf{x}) = \begin{cases} +1, & \text{for } \mathbf{x} = \mathbf{x}_k \text{ and } k \text{ is odd and } 1 \leq k \leq M + N \\ -1, & \text{otherwise} \end{cases}.$$

What is $E_{\text{off}}(h, f)$?

- (b) We say that a target function f can 'generate' $\mathcal{D}$ in a noiseless setting if $y_n = f(\mathbf{x}_n)$ for all $(\mathbf{x}_n, y_n) \in \mathcal{D}$. For a fixed $\mathcal{D}$ of size N , how many possible $f: \mathcal{X} \rightarrow \mathcal{Y}$ can generate $\mathcal{D}$ in a noiseless setting?
- (c) For a given hypothesis h and an integer k between 0 and M , how many of those f in (b) satisfy $E_{\text{off}}(h, f) = \frac{k}{M}$?
- (d) For a given hypothesis h , if all those f that generate $\mathcal{D}$ in a noiseless setting are equally likely in probability, what is the expected off training-set error $\mathbb{E}_f[E_{\text{off}}(h, f)]$?

(continued on next page)

- (e) A deterministic algorithm A is defined as a procedure that takes $\mathcal{D}$ as an input, and outputs a hypothesis $h = A(\mathcal{D})$. Argue that for any two deterministic algorithms A_1 and A_2 ,

$$\mathbb{E}_f [E_{\text{off}}(A_1(\mathcal{D}), f)] = \mathbb{E}_f [E_{\text{off}}(A_2(\mathcal{D}), f)].$$

You have now proved that in a noiseless setting, for a fixed $\mathcal{D}$, if all possible f are equally likely, any two deterministic algorithms are equivalent in terms of the expected off training set error. Similar results can be proved for more general settings.

Problem 1.11 The matrix which tabulates the cost of various errors for the CIA and Supermarket applications in Example 1.1 is called a *risk* or *loss matrix*.

For the two risk matrices in Example 1.1, explicitly write down the in sample error E_{in} that one should minimize to obtain g . This in-sample error should weight the different types of errors based on the risk matrix. [Hint: Consider $y_n = +1$ and $y_n = -1$ separately.]

Problem 1.12 This problem investigates how changing the error measure can change the result of the learning process. You have N data points $y_1 \leq \dots \leq y_N$ and wish to estimate a 'representative' value.

- (a) If your algorithm is to find the hypothesis h that minimizes the in sample sum of squared deviations,

$$E_{\text{in}}(h) = \sum_{n=1}^N (h - y_n)^2,$$

then show that your estimate will be the in sample mean,

$$h_{\text{mean}} = \frac{1}{N} \sum_{n=1}^N y_n.$$

- (b) If your algorithm is to find the hypothesis h that minimizes the in sample sum of absolute deviations,

$$E_{\text{in}}(h) = \sum_{n=1}^N |h - y_n|,$$

then show that your estimate will be the in sample median h_{med} , which is any value for which half the data points are at most h_{med} and half the data points are at least h_{med} .

- (c) Suppose y_N is perturbed to $y_N + \epsilon$, where $\epsilon \rightarrow \infty$. So, the single data point y_N becomes an outlier. What happens to your two estimators h_{mean} and h_{med} ?

Chapter 2

Training versus Testing

Before the final exam, a professor may hand out some practice problems and solutions to the class. Although these problems are not the exact ones that will appear on the exam, studying them will help you do better. They are the ‘training set’ in your learning.

If the professor’s goal is to help you do better in the exam, why not give out the exam problems themselves? Well, nice try 😊. Doing well in the exam is not the goal in and of itself. The goal is for you to learn the course material. The exam is merely a way to gauge how well you have learned the material. If the exam problems are known ahead of time, your performance on them will no longer accurately gauge how well you have learned.

The same distinction between training and testing happens in learning from data. In this chapter, we will develop a mathematical theory that characterizes this distinction. We will also discuss the conceptual and practical implications of the contrast between training and testing.

2.1 Theory of Generalization

The out-of-sample error E_{out} measures how well our training on $\mathcal{D}$ has *generalized* to data that we have not seen before. E_{out} is based on the performance over the entire input space $\mathcal{X}$. Intuitively, if we want to estimate the value of E_{out} using a sample of data points, these points must be ‘fresh’ test points that have not been used for training, similar to the questions on the final exam that have not been used for practice.

The in sample error E_{in} , by contrast, is based on data points that have been used for training. It expressly measures training performance, similar to your performance on the practice problems that you got before the final exam. Such performance has the benefit of looking at the solutions and adjusting accordingly, and may not reflect the ultimate performance in a real test. We began the analysis of in-sample error in Chapter 1, and we will extend this

analysis to the general case in this chapter. We will also make the contrast between a training set and a test set more precise.

A word of warning: this chapter is the heaviest in this book in terms of mathematical abstraction. To make it easier on the not-so-mathematically inclined, we will tell you which part you can safely skip without ‘losing the plot’. The mathematical results provide fundamental insights into learning from data, and we will interpret these results in practical terms.

Generalization error. We have already discussed how the value of E_{in} does not always generalize to a similar value of E_{out} . Generalization is a key issue in learning. One can define the *generalization error* as the discrepancy between E_{in} and E_{out} .¹ The Hoeffding Inequality (1.6) provides a way to characterize the generalization error with a probabilistic bound,

$$\mathbb{P}[|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon] \leq 2Me^{-2\epsilon^2 N},$$

for any $\epsilon > 0$. This can be rephrased as follows. Pick a tolerance level δ , for example $\delta = 0.05$, and assert with probability at least $1 - \delta$ that

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{1}{2N} \ln \frac{2M}{\delta}}. \quad (2.1)$$

We refer to the type of inequality in (2.1) as a *generalization bound* because it bounds E_{out} in terms of E_{in} . To see that the Hoeffding Inequality implies this generalization bound, we rewrite (1.6) as follows: with probability at least $1 - 2Me^{-2N\epsilon^2}$, $|E_{\text{out}} - E_{\text{in}}| \leq \epsilon$, which implies $E_{\text{out}} \leq E_{\text{in}} + \epsilon$. We may now identify $\delta = 2Me^{-2N\epsilon^2}$, from which $\epsilon = \sqrt{\frac{1}{2N} \ln \frac{2M}{\delta}}$, and (2.1) follows.

Notice that the other side of $|E_{\text{out}} - E_{\text{in}}| \leq \epsilon$ also holds, that is, $E_{\text{out}} \geq E_{\text{in}} - \epsilon$ for all $h \in \mathcal{H}$. This is important for learning, but in a more subtle way. Not only do we want to know that the hypothesis g that we choose (say the one with the best training error) will continue to do well out of sample (i.e., $E_{\text{out}} \leq E_{\text{in}} + \epsilon$), but we also want to be sure that we did the best we could with our $\mathcal{H}$ (no other hypothesis $h \in \mathcal{H}$ has $E_{\text{out}}(h)$ significantly better than $E_{\text{out}}(g)$). The $E_{\text{out}}(h) \geq E_{\text{in}}(h) - \epsilon$ direction of the bound assures us that we couldn’t do much better because every hypothesis with a higher E_{in} than the g we have chosen will have a comparably higher E_{out} .

The error bound $\sqrt{\frac{1}{2N} \ln \frac{2M}{\delta}}$ in (2.1), or ‘error bar’ if you will, depends on M , the size of the hypothesis set $\mathcal{H}$. If $\mathcal{H}$ is an infinite set, the bound goes to infinity and becomes meaningless. Unfortunately, almost all interesting learning models have infinite $\mathcal{H}$, including the simple perceptron which we discussed in Chapter 1.

In order to study generalization in such models, we need to derive a counterpart to (2.1) that deals with infinite $\mathcal{H}$. We would like to replace M with

¹Sometimes ‘generalization error’ is used as another name for E_{out} , but not in this book.

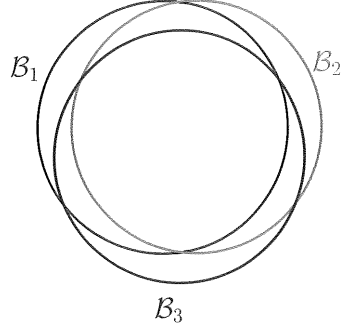
something finite, so that the bound is meaningful. To do this, we notice that the way we got the M factor in the first place was by taking the disjunction of events:

$$\begin{aligned}
 & \text{"}|E_{\text{in}}(h_1) - E_{\text{out}}(h_1)| > \epsilon" \quad \text{or} \\
 & \text{"}|E_{\text{in}}(h_2) - E_{\text{out}}(h_2)| > \epsilon" \quad \text{or} \\
 & \quad \vdots \\
 & \text{"}|E_{\text{in}}(h_M) - E_{\text{out}}(h_M)| > \epsilon" , \tag{2.2}
 \end{aligned}$$

which is guaranteed to include the event " $|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon$ " since g is always one of the hypotheses in $\mathcal{H}$. We then over-estimated the probability using the union bound. Let $\mathcal{B}_m$ be the ($\mathcal{B}$ ad) event that " $|E_{\text{in}}(h_m) - E_{\text{out}}(h_m)| > \epsilon$ ". Then,

$$\mathbb{P}[\mathcal{B}_1 \text{ or } \mathcal{B}_2 \text{ or } \cdots \text{ or } \mathcal{B}_M] \leq \mathbb{P}[\mathcal{B}_1] + \mathbb{P}[\mathcal{B}_2] + \cdots + \mathbb{P}[\mathcal{B}_M].$$

If the events $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_M$ are strongly overlapping, the union bound becomes particularly loose as illustrated in the figure to the right for an example with 3 hypotheses; the areas of different events correspond to their probabilities. The union bound says that the total area covered by $\mathcal{B}_1, \mathcal{B}_2$, or $\mathcal{B}_3$ is smaller than the sum of the individual areas, which is true but is a gross overestimate when the areas overlap heavily as in this example. The events " $|E_{\text{in}}(h_m) - E_{\text{out}}(h_m)| > \epsilon$ "; $m = 1, \dots, M$, are often strongly overlapping. If h_1 is very similar to h_2 for instance, the two events " $|E_{\text{in}}(h_1) - E_{\text{out}}(h_1)| > \epsilon$ " and " $|E_{\text{in}}(h_2) - E_{\text{out}}(h_2)| > \epsilon$ " are likely to coincide for most data sets. In a typical learning model, many hypotheses are indeed very similar. If you take the perceptron model for instance, as you slowly vary the weight vector $\mathbf{w}$, you get infinitely many hypotheses that differ from each other only infinitesimally.



The mathematical theory of generalization hinges on this observation. Once we properly account for the overlaps of the different hypotheses, we will be able to replace the number of hypotheses M in (2.1) by an effective number which is finite even when M is infinite, and establish a more useful condition under which E_{out} is close to E_{in} .

2.1.1 Effective Number of Hypotheses

We now introduce the *growth function*, the quantity that will formalize the effective number of hypotheses. The growth function is what will replace M

in the generalization bound (2.1). It is a combinatorial quantity that captures how different the hypotheses in $\mathcal{H}$ are, and hence how much overlap the different events in (2.2) have.

We will start by defining the growth function and studying its basic properties. Next, we will show how we can bound the value of the growth function. Finally, we will show that we can replace M in the generalization bound with the growth function. These three steps will yield the generalization bound that we need, which applies to infinite $\mathcal{H}$. We will focus on binary target functions for the purpose of this analysis, so each $h \in \mathcal{H}$ maps $\mathcal{X}$ to $\{-1, +1\}$.

The definition of the growth function is based on the number of different hypotheses that $\mathcal{H}$ can implement, but only over a finite sample of points rather than over the entire input space $\mathcal{X}$. If $h \in \mathcal{H}$ is applied to a finite sample $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathcal{X}$, we get an N -tuple $h(\mathbf{x}_1), \dots, h(\mathbf{x}_N)$ of ± 1 's. Such an N -tuple is called a *dichotomy* since it splits $\mathbf{x}_1, \dots, \mathbf{x}_N$ into two groups: those points for which h is -1 and those for which h is $+1$. Each $h \in \mathcal{H}$ generates a dichotomy on $\mathbf{x}_1, \dots, \mathbf{x}_N$, but two different h 's may generate the same dichotomy if they happen to give the same pattern of ± 1 's on this particular sample.

Definition 2.1. Let $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathcal{X}$. The dichotomies generated by $\mathcal{H}$ on these points are defined by

$$\mathcal{H}(\mathbf{x}_1, \dots, \mathbf{x}_N) = \{ (h(\mathbf{x}_1), \dots, h(\mathbf{x}_N)) \mid h \in \mathcal{H} \}. \quad (2.3)$$

One can think of the dichotomies $\mathcal{H}(\mathbf{x}_1, \dots, \mathbf{x}_N)$ as a set of hypotheses just like $\mathcal{H}$ is, except that the hypotheses are seen through the eyes of N points only. A larger $\mathcal{H}(\mathbf{x}_1, \dots, \mathbf{x}_N)$ means $\mathcal{H}$ is more 'diverse' generating more dichotomies on $\mathbf{x}_1, \dots, \mathbf{x}_N$. The growth function is based on the number of dichotomies.

Definition 2.2. The growth function is defined for a hypothesis set $\mathcal{H}$ by

$$m_{\mathcal{H}}(N) = \max_{\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathcal{X}} |\mathcal{H}(\mathbf{x}_1, \dots, \mathbf{x}_N)|,$$

where $|\cdot|$ denotes the cardinality (number of elements) of a set.

In words, $m_{\mathcal{H}}(N)$ is the maximum number of dichotomies that can be generated by $\mathcal{H}$ on any N points. To compute $m_{\mathcal{H}}(N)$, we consider all possible choices of N points $\mathbf{x}_1, \dots, \mathbf{x}_N$ from $\mathcal{X}$ and pick the one that gives us the most dichotomies. Like M , $m_{\mathcal{H}}(N)$ is a measure of the number of hypotheses in $\mathcal{H}$, except that a hypothesis is now considered on N points instead of the entire $\mathcal{X}$. For any $\mathcal{H}$, since $\mathcal{H}(\mathbf{x}_1, \dots, \mathbf{x}_N) \subseteq \{-1, +1\}^N$ (the set of all possible dichotomies on any N points), the value of $m_{\mathcal{H}}(N)$ is at most $|\{-1, +1\}^N|$, hence

$$m_{\mathcal{H}}(N) \leq 2^N.$$

If $\mathcal{H}$ is capable of generating all possible dichotomies on $\mathbf{x}_1, \dots, \mathbf{x}_N$, then $\mathcal{H}(\mathbf{x}_1, \dots, \mathbf{x}_N) = \{-1, +1\}^N$ and we say that $\mathcal{H}$ can *shatter* $\mathbf{x}_1, \dots, \mathbf{x}_N$. This signifies that $\mathcal{H}$ is as diverse as can be on this particular sample.

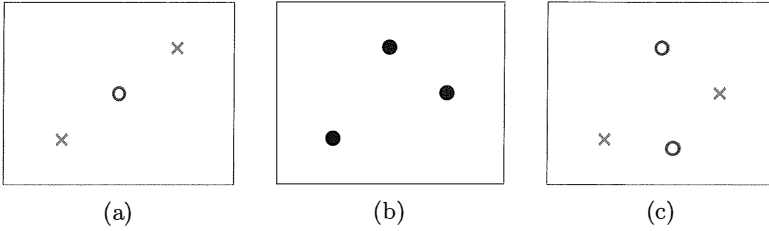


Figure 2.1: Illustration of the growth function for a two dimensional perceptron. The dichotomy of red versus blue on the 3 colinear points in part (a) cannot be generated by a perceptron, but all 8 dichotomies on the 3 points in part (b) can. By contrast, the dichotomy of red versus blue on the 4 points in part (c) cannot be generated by a perceptron. At most 14 out of the possible 16 dichotomies on any 4 points can be generated.

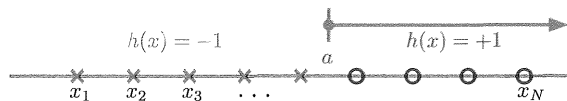
Example 2.1. If $\mathcal{X}$ is a Euclidean plane and $\mathcal{H}$ is a two-dimensional perceptron, what are $m_{\mathcal{H}}(3)$ and $m_{\mathcal{H}}(4)$? Figure 2.1(a) shows a dichotomy on 3 points that the perceptron cannot generate, while Figure 2.1(b) shows another 3 points that the perceptron can shatter, generating all $2^3 = 8$ dichotomies. Because the definition of $m_{\mathcal{H}}(N)$ is based on the maximum number of dichotomies, $m_{\mathcal{H}}(3) = 8$ in spite of the case in Figure 2.1(a).

In the case of 4 points, Figure 2.1(c) shows a dichotomy that the perceptron cannot generate. One can verify that there are no 4 points that the perceptron can shatter. The most a perceptron can do on any 4 points is 14 dichotomies out of the possible 16, where the 2 missing dichotomies are as depicted in Figure 2.1(c) with blue and red corresponding to -1 , $+1$ or to $+1$, -1 . Hence, $m_{\mathcal{H}}(4) = 14$. $\square$

Let us now illustrate how to compute $m_{\mathcal{H}}(N)$ for some simple hypothesis sets. These examples will confirm the intuition that $m_{\mathcal{H}}(N)$ grows faster when the hypothesis set $\mathcal{H}$ becomes more complex. This is what we expect of a quantity that is meant to replace M in the generalization bound (2.1).

Example 2.2. Let us find a formula for $m_{\mathcal{H}}(N)$ in each of the following cases.

1. *Positive rays:* $\mathcal{H}$ consists of all hypotheses $h: \mathbb{R} \rightarrow \{-1, +1\}$ of the form $h(x) = \text{sign}(x - a)$, i.e., the hypotheses are defined in a one-dimensional input space, and they return -1 to the left of some value a and $+1$ to the right of a .

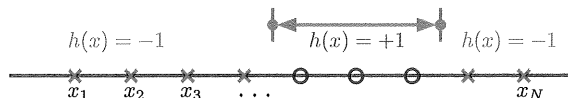


To compute $m_{\mathcal{H}}(N)$, we notice that given N points, the line is split by the points into $N + 1$ regions. The dichotomy we get on the N points is decided by which region contains the value a . As we vary a , we will get $N + 1$ different dichotomies. Since this is the most we can get for any N points, the growth function is

$$m_{\mathcal{H}}(N) = N + 1.$$

Notice that if we picked N points where some of the points coincided (which is allowed), we will get less than $N + 1$ dichotomies. This does not affect the value of $m_{\mathcal{H}}(N)$ since it is defined based on the maximum number of dichotomies.

2. *Positive intervals*: $\mathcal{H}$ consists of all hypotheses in one dimension that return $+1$ within some interval and -1 otherwise. Each hypothesis is specified by the two end values of that interval.



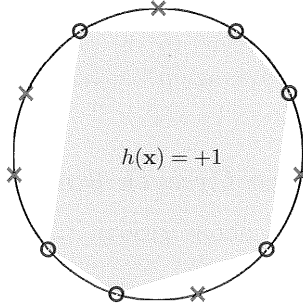
To compute $m_{\mathcal{H}}(N)$, we notice that given N points, the line is again split by the points into $N + 1$ regions. The dichotomy we get is decided by which two regions contain the end values of the interval, resulting in $\binom{N+1}{2}$ different dichotomies. If both end values fall in the same region, the resulting hypothesis is the constant -1 regardless of which region it is. Adding up these possibilities, we get

$$m_{\mathcal{H}}(N) = \binom{N+1}{2} + 1 = \frac{1}{2}N^2 + \frac{1}{2}N + 1.$$

Notice that $m_{\mathcal{H}}(N)$ grows as the square of N , faster than the linear $m_{\mathcal{H}}(N)$ of the ‘simpler’ positive ray case.

3. *Convex sets*: $\mathcal{H}$ consists of all hypotheses in two dimensions $h: \mathbb{R}^2 \rightarrow \{-1, +1\}$ that are positive inside some convex set and negative elsewhere (a set is convex if the line segment connecting any two points in the set lies entirely within the set).

To compute $m_{\mathcal{H}}(N)$ in this case, we need to choose the N points carefully. Per the next figure, choose N points on the perimeter of a circle. Now consider any dichotomy on these points, assigning an arbitrary pattern of ± 1 ’s to the N points. If you connect the $+1$ points with a polygon, the hypothesis made up of the closed interior of the polygon (which has to be convex since its vertices are on the perimeter of a circle) agrees with the dichotomy on all N points. For the dichotomies that have less than three $+1$ points, the convex set will be a line segment, a point, or an empty set.



This means that any dichotomy on these N points can be realized using a convex hypothesis, so $\mathcal{H}$ manages to shatter these points and the growth function has the maximum possible value

$$m_{\mathcal{H}}(N) = 2^N.$$

Notice that if the N points were chosen at random in the plane rather than on the perimeter of a circle, many of the points would be ‘internal’ and we wouldn’t be able to shatter all the points with convex hypotheses as we did for the perimeter points. However, this doesn’t matter as far as $m_{\mathcal{H}}(N)$ is concerned, since it is defined based on the maximum (2^N in this case). $\square$

It is not practical to try to compute $m_{\mathcal{H}}(N)$ for every hypothesis set we use. Fortunately, we don’t have to. Since $m_{\mathcal{H}}(N)$ is meant to replace M in (2.1), we can use an upper bound on $m_{\mathcal{H}}(N)$ instead of the exact value, and the inequality in (2.1) will still hold. Getting a good bound on $m_{\mathcal{H}}(N)$ will prove much easier than computing $m_{\mathcal{H}}(N)$ itself, thanks to the notion of a *break point*.

Definition 2.3. *If no data set of size k can be shattered by $\mathcal{H}$, then k is said to be a break point for $\mathcal{H}$.*

If k is a break point, then $m_{\mathcal{H}}(k) < 2^k$. Example 2.1 shows that $k = 4$ is a break point for two-dimensional perceptrons. In general, it is easier to find a break point for $\mathcal{H}$ than to compute the full growth function for that $\mathcal{H}$.

Exercise 2.1

By inspection, find a break point k for each hypothesis set in Example 2.2 (if there is one). Verify that $m_{\mathcal{H}}(k) < 2^k$ using the formulas derived in that Example.

We now use the break point k to derive a bound on the growth function $m_{\mathcal{H}}(N)$ for all values of N . For example, the fact that no 4 points can be shattered by

the two-dimensional perceptron puts a significant constraint on the number of dichotomies that can be realized by the perceptron on 5 or more points. We will exploit this idea to get a significant bound on $m_{\mathcal{H}}(N)$ in general.

2.1.2 Bounding the Growth Function

The most important fact about growth functions is that if the condition $m_{\mathcal{H}}(N) = 2^N$ breaks at any point, we can bound $m_{\mathcal{H}}(N)$ for all values of N by a simple polynomial based on this break point. The fact that the bound is polynomial is crucial. Absent a break point (as is the case in the convex hypothesis example), $m_{\mathcal{H}}(N) = 2^N$ for all N . If $m_{\mathcal{H}}(N)$ replaced M in Equation (2.1), the bound $\sqrt{\frac{1}{2N} \ln \frac{2M}{\delta}}$ on the generalization error would not go to zero regardless of how many training examples N we have. However, if $m_{\mathcal{H}}(N)$ can be bounded by a polynomial — any polynomial —, the generalization error will go to zero as $N \rightarrow \infty$. This means that we will generalize well given a sufficient number of examples.

Begin safe skip: If you trust our math, you can skip the following part without compromising the logical sequence. A similar green box will tell you when to rejoin.

To prove the polynomial bound, we will introduce a combinatorial quantity that counts the maximum number of dichotomies given that there is a break point, without having to assume any particular form of $\mathcal{H}$. This bound will therefore apply to any $\mathcal{H}$.

Definition 2.4. $B(N, k)$ is the maximum number of dichotomies on N points such that no subset of size k of the N points can be shattered by these dichotomies.

The definition of $B(N, k)$ assumes a break point k , then tries to find the most dichotomies on N points without imposing any further restrictions. Since $B(N, k)$ is defined as a maximum, it will serve as an upper bound for any $m_{\mathcal{H}}(N)$ that has a break point k ;

$$m_{\mathcal{H}}(N) \leq B(N, k) \quad \text{if } k \text{ is a break point for } \mathcal{H}.$$

The notation B comes from ‘Binomial’ and the reason will become clear shortly. To evaluate $B(N, k)$, we start with the two boundary conditions $k = 1$ and $N = 1$.

$$\begin{aligned} B(N, 1) &= 1 \\ B(1, k) &= 2 \quad \text{for } k > 1. \end{aligned}$$

$B(N, 1) = 1$ for all N since if no subset of size 1 can be shattered, then only one dichotomy can be allowed. A second different dichotomy must differ on at least one point and then that subset of size 1 would be shattered. $B(1, k) = 2$ for $k > 1$ since in this case there do not even exist subsets of size k ; the constraint is vacuously true and we have 2 possible dichotomies (+1 and -1) on the one point.

We now assume $N \geq 2$ and $k \geq 2$ and try to develop a recursion. Consider the $B(N, k)$ dichotomies in definition 2.4, where no k points can be shattered. We list these dichotomies in the following table,

	# of rows	$\mathbf{x}_1$	$\mathbf{x}_2$	$\dots$	$\mathbf{x}_{N-1}$	$\mathbf{x}_N$
S_1	α	+1	+1	$\dots$	+1	+1
		-1	+1	$\dots$	+1	-1
		$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
		+1	-1	$\dots$	-1	-1
		-1	+1	$\dots$	-1	+1
S_2	β	+1	-1	$\dots$	+1	+1
		-1	-1	$\dots$	+1	+1
		$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
		+1	-1	$\dots$	+1	+1
		-1	-1	$\dots$	-1	+1
S_2^-	β	+1	-1	$\dots$	+1	-1
		-1	-1	$\dots$	+1	-1
		$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
		+1	-1	$\dots$	+1	-1
		-1	-1	$\dots$	-1	-1

where $\mathbf{x}_1, \dots, \mathbf{x}_N$ in the table are labels for the N points of the dichotomy. We have chosen a convenient order in which to list the dichotomies, as follows. Consider the dichotomies on $\mathbf{x}_1, \dots, \mathbf{x}_{N-1}$. Some dichotomies on these $N-1$ points appear only once (with either +1 or -1 in the $\mathbf{x}_N$ column, but not both). We collect these dichotomies in the set S_1 . The remaining dichotomies on the first $N-1$ points appear twice, once with +1 and once with -1 in the $\mathbf{x}_N$ column. We collect these dichotomies in the set S_2 which can be divided into two equal parts, S_2^+ and S_2^- (with +1 and -1 in the $\mathbf{x}_N$ column, respectively). Let S_1 have α rows, and let S_2^+ and S_2^- have β rows each. Since the total number of rows in the table is $B(N, k)$ by construction, we have

$$B(N, k) = \alpha + 2\beta. \quad (2.4)$$

The total number of different dichotomies on the first $N-1$ points is given by $\alpha + \beta$; since S_2^+ and S_2^- are identical on these $N-1$ points, their dichotomies are redundant. Since no subset of k of these first $N-1$ points can

be shattered (since no k -subset of all N points can be shattered), we deduce that

$$\alpha + \beta \leq B(N-1, k) \quad (2.5)$$

by definition of B . Further, no subset of size $k-1$ of the first $N-1$ points can be shattered by the dichotomies in S_2^+ . If there existed such a subset, then taking the corresponding set of dichotomies in S_2^- and adding $\mathbf{x}_N$ to the data points yields a subset of size k that is shattered, which we know cannot exist in this table by definition of $B(N, k)$. Therefore,

$$\beta \leq B(N-1, k-1). \quad (2.6)$$

Substituting the two Inequalities (2.5) and (2.6) into (2.4), we get

$$B(N, k) \leq B(N-1, k) + B(N-1, k-1). \quad (2.7)$$

We can use (2.7) to recursively compute a bound on $B(N, k)$, as shown in the following table.

		k						
		1	2	3	4	5	6	...
N	1	1	2	2	2	2	2	...
	2	1	3	4	4	4	4	...
	3	1	4	7	8	8	8	...
	4	1	5	11	...	...	...	...
	5	1	6	$\vdots$	$\ddots$			
	6	1	7	$\vdots$		$\ddots$		
	$\vdots$	$\vdots$	$\vdots$	$\vdots$			$\ddots$	

where the first row ($N = 1$) and the first column ($k = 1$) are the boundary conditions that we already calculated. We can also use the recursion to bound $B(N, k)$ analytically.

Lemma 2.3 (Sauer's Lemma).

$$B(N, k) \leq \sum_{i=0}^{k-1} \binom{N}{i}$$

Proof. The statement is true whenever $k = 1$ or $N = 1$, by inspection. The proof is by induction on N . Assume the statement is true for all $N \leq N_o$ and all k . We need to prove the statement for $N = N_o + 1$ and all k . Since the statement is already true when $k = 1$ (for all values of N) by the initial condition, we only need to worry about $k \geq 2$. By (2.7),

$$B(N_o + 1, k) \leq B(N_o, k) + B(N_o, k-1).$$

Applying the induction hypothesis to each term on the RHS, we get

$$\begin{aligned}
 B(N_o + 1, k) &\leq \sum_{i=0}^{k-1} \binom{N_o}{i} + \sum_{i=0}^{k-2} \binom{N_o}{i} \\
 &= 1 + \sum_{i=1}^{k-1} \binom{N_o}{i} + \sum_{i=1}^{k-1} \binom{N_o}{i-1} \\
 &= 1 + \sum_{i=1}^{k-1} \left[\binom{N_o}{i} + \binom{N_o}{i-1} \right] \\
 &= 1 + \sum_{i=1}^{k-1} \binom{N_o + 1}{i} = \sum_{i=0}^{k-1} \binom{N_o + 1}{i},
 \end{aligned}$$

where the combinatorial identity $\binom{N_o+1}{i} = \binom{N_o}{i} + \binom{N_o}{i-1}$ has been used. This identity can be proved by noticing that to calculate the number of ways to pick i objects from N_o+1 distinct objects, either the first object is included, in $\binom{N_o}{i-1}$ ways, or the first object is not included, in $\binom{N_o}{i}$ ways. We have thus proved the induction step, so the statement is true for all N and k . ■

It turns out that $B(N, k)$ in fact equals $\sum_{i=0}^{k-1} \binom{N}{i}$ (see Problem 2.4), but we only need the inequality of Lemma 2.3 to bound the growth function. For a given break point k , the bound $\sum_{i=0}^{k-1} \binom{N}{i}$ is polynomial in N , as each term in the sum is polynomial (of degree $i \leq k-1$). Since $B(N, k)$ is an upper bound on any $m_{\mathcal{H}}(N)$ that has a break point k , we have proved

End safe skip: Those who skipped are now rejoining us. The next theorem states that any growth function $m_{\mathcal{H}}(N)$ with a break point is bounded by a polynomial.

Theorem 2.4. If $m_{\mathcal{H}}(k) < 2^k$ for some value k , then

$$m_{\mathcal{H}}(N) \leq \sum_{i=0}^{k-1} \binom{N}{i} \quad (2.8)$$

for all N . The RHS is polynomial in N of degree $k-1$.

The implication of Theorem 2.4 is that if $\mathcal{H}$ has a break point, we have what we want to ensure good generalization; a polynomial bound on $m_{\mathcal{H}}(N)$.

Exercise 2.2

- (a) Verify the bound of Theorem 2.4 in the three cases of Example 2.2:
- (i) Positive rays: $\mathcal{H}$ consists of all hypotheses in one dimension of the form $h(x) = \text{sign}(x - a)$.
 - (ii) Positive intervals: $\mathcal{H}$ consists of all hypotheses in one dimension that are positive within some interval and negative elsewhere.
 - (iii) Convex sets: $\mathcal{H}$ consists of all hypotheses in two dimensions that are positive inside some convex set and negative elsewhere.
- (Note: you can use the break points you found in Exercise 2.1.)
- (b) Does there exist a hypothesis set for which $m_{\mathcal{H}}(N) = N + 2^{\lfloor N/2 \rfloor}$ (where $\lfloor N/2 \rfloor$ is the largest integer $\leq N/2$)?

2.1.3 The VC Dimension

Theorem 2.4 bounds the entire growth function in terms of any break point. The smaller the break point, the better the bound. This leads us to the following definition of a single parameter that characterizes the growth function.

Definition 2.5. *The Vapnik-Chervonenkis dimension of a hypothesis set $\mathcal{H}$, denoted by $d_{\text{VC}}(\mathcal{H})$ or simply d_{VC} , is the largest value of N for which $m_{\mathcal{H}}(N) = 2^N$. If $m_{\mathcal{H}}(N) = 2^N$ for all N , then $d_{\text{VC}}(\mathcal{H}) = \infty$.*

If d_{VC} is the VC dimension of $\mathcal{H}$, then $k = d_{\text{VC}} + 1$ is a break point for $m_{\mathcal{H}}$ since $m_{\mathcal{H}}(N)$ cannot equal 2^N for any $N > d_{\text{VC}}$ by definition. It is easy to see that no smaller break point exists since $\mathcal{H}$ can shatter d_{VC} points, hence it can also shatter any subset of these points.

Exercise 2.3

Compute the VC dimension of $\mathcal{H}$ for the hypothesis sets in parts (i), (ii), (iii) of Exercise 2.2(a).

Since $k = d_{\text{VC}} + 1$ is a break point for $m_{\mathcal{H}}$, Theorem 2.4 can be rewritten in terms of the VC dimension:

$$m_{\mathcal{H}}(N) \leq \sum_{i=0}^{d_{\text{VC}}} \binom{N}{i}. \quad (2.9)$$

Therefore, the VC dimension is the order of the polynomial bound on $m_{\mathcal{H}}(N)$. It is also the best we can do using this line of reasoning, because no smaller break point than $k = d_{\text{VC}} + 1$ exists. The form of the polynomial bound can be further simplified to make the dependency on d_{VC} more salient. We state a useful form here, which can be proved by induction (Problem 2.5).

$$m_{\mathcal{H}}(N) \leq N^{d_{\text{VC}}} + 1. \quad (2.10)$$

Now that the growth function has been bounded in terms of the VC dimension, we have only one more step left in our analysis, which is to replace the number of hypotheses M in the generalization bound (2.1) with the growth function $m_{\mathcal{H}}(N)$. If we manage to do that, the VC dimension will play a pivotal role in the generalization question. If we were to directly replace M by $m_{\mathcal{H}}(N)$ in (2.1), we would get a bound of the form

$$E_{\text{out}} \stackrel{?}{\leq} E_{\text{in}} + \sqrt{\frac{1}{2N} \ln \frac{2m_{\mathcal{H}}(N)}{\delta}}.$$

Unless $d_{\text{VC}}(\mathcal{H}) = \infty$, we know that $m_{\mathcal{H}}(N)$ is bounded by a polynomial in N ; thus, $\ln m_{\mathcal{H}}(N)$ grows logarithmically in N regardless of the order of the polynomial, and so it will be crushed by the $\frac{1}{N}$ factor. Therefore, for any fixed tolerance δ , the bound on E_{out} will be arbitrarily close to E_{in} for sufficiently large N .

Only if $d_{\text{VC}}(\mathcal{H}) = \infty$ will this argument fail, as the growth function in this case is exponential in N . For any finite value of d_{VC} , the error bar will converge to zero at a speed determined by d_{VC} , since d_{VC} is the order of the polynomial. The smaller d_{VC} is, the faster the convergence to zero.

It turns out that we cannot just replace M with $m_{\mathcal{H}}(N)$ in the generalization bound (2.1), but rather we need to make other adjustments as we will see shortly. However, the general idea above is correct, and d_{VC} will still play the role that we discussed here. One implication of this discussion is that there is a division of models into two classes. The ‘good models’ have finite d_{VC} , and for sufficiently large N , E_{in} will be close to E_{out} ; for good models, the in-sample performance generalizes to out of sample. The ‘bad models’ have infinite d_{VC} . With a bad model, no matter how large the data set is, we cannot make generalization conclusions from E_{in} to E_{out} based on the VC analysis.²

Because of its significant role, it is worthwhile to try to gain some insight about the VC dimension before we proceed to the formalities of deriving the new generalization bound. One way to gain insight about d_{VC} is to try to compute it for learning models that we are familiar with. Perceptrons are one case where we can compute d_{VC} exactly. This is done in two steps. First, we show that d_{VC} is at least a certain value, then we show that it is at most the same value. There is a logical difference in arguing that d_{VC} is at least a certain value, as opposed to at most a certain value. This is because

$$d_{\text{VC}} \geq N \iff \text{there exists } \mathcal{D} \text{ of size } N \text{ such that } \mathcal{H} \text{ shatters } \mathcal{D},$$

hence we have different conclusions in the following cases.

1. There is a set of N points that can be shattered by $\mathcal{H}$. In this case, we can conclude that $d_{\text{VC}} \geq N$.

²In some cases with infinite d_{VC} , such as the convex sets that we discussed, alternative analysis based on an ‘average’ growth function can establish good generalization behavior.

2. Any set of N points can be shattered by $\mathcal{H}$. In this case, we have more than enough information to conclude that $d_{\text{VC}} \geq N$.
3. There is a set of N points that cannot be shattered by $\mathcal{H}$. Based only on this information, we cannot conclude anything about the value of d_{VC} .
4. No set of N points can be shattered by $\mathcal{H}$. In this case, we can conclude that $d_{\text{VC}} < N$.

Exercise 2.4

Consider the input space $\mathcal{X} = \{1\} \times \mathbb{R}^d$ (including the constant coordinate $x_0 = 1$). Show that the VC dimension of the perceptron (with $d + 1$ parameters, counting w_0) is exactly $d + 1$ by showing that it is at least $d + 1$ and at most $d + 1$, as follows.

- (a) To show that $d_{\text{VC}} \geq d + 1$, find $d + 1$ points in $\mathcal{X}$ that the perceptron can shatter. *[Hint: Construct a nonsingular $(d + 1) \times (d + 1)$ matrix whose rows represent the $d + 1$ points, then use the nonsingularity to argue that the perceptron can shatter these points.]*
- (b) To show that $d_{\text{VC}} \leq d + 1$, show that no set of $d + 2$ points in $\mathcal{X}$ can be shattered by the perceptron. *[Hint: Represent each point in $\mathcal{X}$ as a vector of length $d + 1$, then use the fact that any $d + 2$ vectors of length $d + 1$ have to be linearly dependent. This means that some vector is a linear combination of all the other vectors. Now, if you choose the class of these other vectors carefully, then the classification of the dependent vector will be dictated. Conclude that there is some dichotomy that cannot be implemented, and therefore that for $N \geq d + 2$, $m_{\mathcal{H}}(N) < 2^N$.]*

The VC dimension of a d -dimensional perceptron³ is indeed $d + 1$. This is consistent with Figure 2.1 for the case $d = 2$, which shows a VC dimension of 3. The perceptron case provides a nice intuition about the VC dimension, since $d + 1$ is also the number of parameters in this model. One can view the VC dimension as measuring the ‘effective’ number of parameters. The more parameters a model has, the more diverse its hypothesis set is, which is reflected in a larger value of the growth function $m_{\mathcal{H}}(N)$. In the case of perceptrons, the effective parameters correspond to explicit parameters in the model, namely $w_0, w_1, \dots, w_d$. In other models, the effective parameters may be less obvious or implicit. The VC dimension measures these effective parameters or ‘degrees of freedom’ that enable the model to express a diverse set of hypotheses.

Diversity is not necessarily a good thing in the context of generalization. For example, the set of all possible hypotheses is as diverse as can be, so $m_{\mathcal{H}}(N) = 2^N$ for all N and $d_{\text{VC}}(\mathcal{H}) = \infty$. In this case, no generalization at all is to be expected, as the final version of the generalization bound will show.

³ $\mathcal{X} = \{1\} \times \mathbb{R}^d$ is considered d dimensional since the first coordinate $x_0 = 1$ is fixed.

2.1.4 The VC Generalization Bound

If we treated the growth function as an effective number of hypotheses, and replaced M in the generalization bound (2.1) with $m_{\mathcal{H}}(N)$, the resulting bound would be

$$E_{\text{out}}(g) \stackrel{?}{\leq} E_{\text{in}}(g) + \sqrt{\frac{1}{2N} \ln \frac{2m_{\mathcal{H}}(N)}{\delta}}. \quad (2.11)$$

It turns out that this is not exactly the form that will hold. The quantities in red need to be technically modified to make (2.11) true. The correct bound, which is called the VC generalization bound, is given in the following theorem; it holds for any binary target function f , any hypothesis set $\mathcal{H}$, any learning algorithm $\mathcal{A}$, and any input probability distribution P .

Theorem 2.5 (VC generalization bound). For any tolerance $\delta > 0$,

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{8}{N} \ln \frac{4m_{\mathcal{H}}(2N)}{\delta}} \quad (2.12)$$

with probability $\geq 1 - \delta$.

If you compare the blue items in (2.12) to their red counterparts in (2.11), you notice that all the blue items move the bound in the weaker direction. However, as long as the VC dimension is finite, the error bar still converges to zero (albeit at a slower rate), since $m_{\mathcal{H}}(2N)$ is also polynomial of order d_{VC} in N , just like $m_{\mathcal{H}}(N)$. This means that, with enough data, *each and every hypothesis* in an infinite $\mathcal{H}$ with a finite VC dimension will generalize well from E_{in} to E_{out} . The key is that the effective number of hypotheses, represented by the finite growth function, has replaced the actual number of hypotheses in the bound.

The VC generalization bound is the most important mathematical result in the theory of learning. It establishes the feasibility of learning with infinite hypothesis sets. Since the formal proof is somewhat lengthy and technical, we illustrate the main ideas in a sketch of the proof, and include the formal proof as an appendix. There are two parts to the proof; the justification that the growth function can replace the number of hypotheses in the first place, and the reason why we had to change the red items in (2.11) into the blue items in (2.12).

Sketch of the proof. The data set $\mathcal{D}$ is the source of randomization in the original Hoeffding Inequality. Consider the space of all possible data sets. Let us think of this space as a ‘canvas’ (Figure 2.2(a)). Each $\mathcal{D}$ is a point on that canvas. The probability of a point is determined by which $\mathbf{x}_n$ ’s in $\mathcal{X}$ happen to be in that particular $\mathcal{D}$, and is calculated based on the distribution P over $\mathcal{X}$. Let’s think of probabilities of different events as areas on that canvas, so the total area of the canvas is 1.

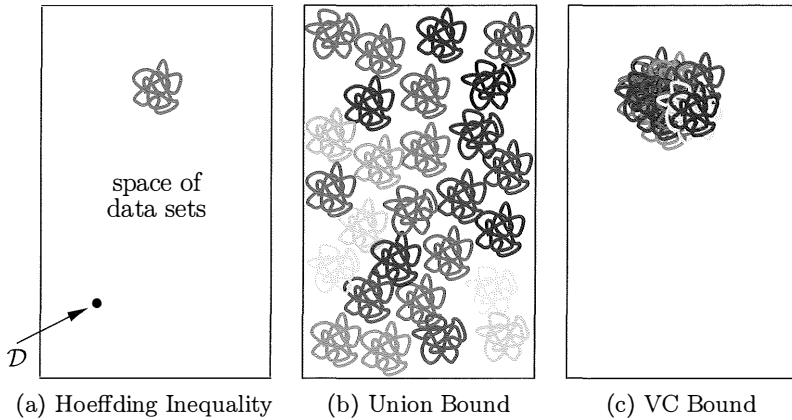


Figure 2.2: Illustration of the proof of the VC bound, where the ‘canvas’ represents the space of all data sets, with areas corresponding to probabilities. (a) For a given hypothesis, the colored points correspond to data sets where E_{in} does not generalize well to E_{out} . The Hoeffding Inequality guarantees a small colored area. (b) For several hypotheses, the union bound assumes no overlaps, so the total colored area is large. (c) The VC bound keeps track of overlaps, so it estimates the total area of bad generalization to be relatively small.

For a given hypothesis $h \in \mathcal{H}$, the event “ $|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon$ ” consists of all points $\mathcal{D}$ for which the statement is true. For a particular h , let us paint all these ‘bad’ points using one color. What the basic Hoeffding Inequality tells us is that the colored area on the canvas will be small (Figure 2.2(a)).

Now, if we take another $h \in \mathcal{H}$, the event “ $|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon$ ” may contain different points, since the event depends on h . Let us paint these points with a different color. The area covered by all the points we colored will be at most the sum of the two individual areas, which is the case only if the two areas have no points in common. This is the worst case that the union bound considers. If we keep throwing in a new colored area for each $h \in \mathcal{H}$, and never overlap with previous colors, the canvas will soon be mostly covered in color (Figure 2.2(b)). Even if each h contributed very little, the sheer number of hypotheses will eventually make the colored area cover the whole canvas. This was the problem with using the union bound in the Hoeffding Inequality (1.6), and not taking the overlaps of the colored areas into consideration.

The bulk of the VC proof deals with how to account for the overlaps. Here is the idea. If you were told that the hypotheses in $\mathcal{H}$ are such that each point on the canvas that is colored will be colored 100 times (because of 100 different h ’s), then the total colored area is now 1/100 of what it would have been if the colored points had not overlapped at all. This is the essence of the VC bound as illustrated in (Figure 2.2(c)). The argument goes as follows.

Many hypotheses share the same dichotomy on a given $\mathcal{D}$, since there are finitely many dichotomies even with an infinite number of hypotheses. Any statement based on $\mathcal{D}$ alone will be simultaneously true or simultaneously false for all the hypotheses that look the same on that particular $\mathcal{D}$. What the growth function enables us to do is to account for this kind of hypothesis redundancy in a precise way, so we can get a factor similar to the ‘100’ in the above example.

When $\mathcal{H}$ is infinite, the redundancy factor will also be infinite since the hypotheses will be divided among a finite number of dichotomies. Therefore, the reduction in the total colored area when we take the redundancy into consideration will be dramatic. If it happens that the number of dichotomies is only a polynomial, the reduction will be so dramatic as to bring the total probability down to a very small value. This is the essence of the proof of Theorem 2.5.

The reason $m_{\mathcal{H}}(2N)$ appears in the VC bound instead of $m_{\mathcal{H}}(N)$ is that the proof uses a sample of $2N$ points instead of N points. Why do we need $2N$ points? The event “ $|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon$ ” depends not only on $\mathcal{D}$, but also on the entire $\mathcal{X}$ because $E_{\text{out}}(h)$ is based on $\mathcal{X}$. This breaks the main premise of grouping h ’s based on their behavior on $\mathcal{D}$, since aspects of each h outside of $\mathcal{D}$ affect the truth of “ $|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon$.” To remedy that, we consider the artificial event “ $|E_{\text{in}}(h) - E'_{\text{in}}(h)| > \epsilon$ ” instead, where E_{in} and E'_{in} are based on two samples $\mathcal{D}$ and $\mathcal{D}'$ each of size N . This is where the $2N$ comes from. It accounts for the total size of the two samples $\mathcal{D}$ and $\mathcal{D}'$. Now, the truth of the statement “ $|E_{\text{in}}(h) - E'_{\text{in}}(h)| > \epsilon$ ” depends exclusively on the total sample of size $2N$, and the above redundancy argument will hold.

Of course we have to justify why the two-sample condition “ $|E_{\text{in}}(h) - E'_{\text{in}}(h)| > \epsilon$ ” can replace the original condition “ $|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon$.” In doing so, we end up having to shrink the ϵ ’s by a factor of 4, and also end up with a factor of 2 in the estimate of the overall probability. This accounts for the $\frac{8}{N}$ instead of $\frac{1}{2N}$ in the VC bound and for having 4 instead of 2 as the multiplicative factor of the growth function. When you put all this together, you get the formula in (2.12). $\square$

2.2 Interpreting the Generalization Bound

The VC generalization bound (2.12) is a universal result in the sense that it applies to all hypothesis sets, learning algorithms, input spaces, probability distributions, and binary target functions. It can be extended to other types of target functions as well. Given the generality of the result, one would suspect that the bound it provides may not be particularly tight in any given case, since the same bound has to cover a lot of different cases. Indeed, the bound is quite loose.

Exercise 2.5

Suppose we have a simple learning model whose growth function is $m_{\mathcal{H}}(N) = N + 1$, hence $d_{\text{VC}} = 1$. Use the VC bound (2.12) to estimate the probability that E_{out} will be within 0.1 of E_{in} given 100 training examples. *[Hint: The estimate will be ridiculous.]*

Why is the VC bound so loose? The slack in the bound can be attributed to a number of technical factors. Among them,

1. The basic Hoeffding Inequality used in the proof already has a slack. The inequality gives the same bound whether E_{out} is close to 0.5 or close to zero. However, the variance of E_{in} is quite different in these two cases. Therefore, having one bound capture both cases will result in some slack.
2. Using $m_{\mathcal{H}}(N)$ to quantify the number of dichotomies on N points, regardless of which N points are in the data set, gives us a worst-case estimate. This does allow the bound to be independent of the probability distribution P over $\mathcal{X}$. However, we would get a more tuned bound if we considered specific $\mathbf{x}_1, \dots, \mathbf{x}_N$ and used $|\mathcal{H}(\mathbf{x}_1, \dots, \mathbf{x}_N)|$ or its expected value instead of the upper bound $m_{\mathcal{H}}(N)$. For instance, in the case of convex sets in two dimensions, which we examined in Example 2.2, if you pick N points at random in the plane, they will likely have far fewer dichotomies than 2^N , while $m_{\mathcal{H}}(N) = 2^N$.
3. Bounding $m_{\mathcal{H}}(N)$ by a simple polynomial of order d_{VC} , as given in (2.10), will contribute further slack to the VC bound.

Some effort could be put into tightening the VC bound, but many highly technical attempts in the literature have resulted in only diminishing returns. The reality is that the VC line of analysis leads to a very loose bound. Why did we bother to go through the analysis then? Two reasons. First, the VC analysis is what establishes the feasibility of learning for infinite hypothesis sets, the only kind we use in practice. Second, although the bound is loose, it tends to be equally loose for different learning models, and hence is useful for comparing the generalization performance of these models. This is an observation from practical experience, not a mathematical statement. In real applications, learning models with lower d_{VC} tend to generalize better than those with higher d_{VC} . Because of this observation, the VC analysis proves useful in practice, and some rules of thumb have emerged in terms of the VC dimension. For instance, requiring that N be at least $10 \times d_{\text{VC}}$ to get decent generalization is a popular rule of thumb.

Thus, the VC bound can be used as a guideline for generalization, relatively if not absolutely. With this understanding, let us look at the different ways the bound is used in practice.

2.2.1 Sample Complexity

The sample complexity denotes how many training examples N are needed to achieve a certain generalization performance. The performance is specified by two parameters, ϵ and δ . The error tolerance ϵ determines the allowed generalization error, and the confidence parameter δ determines how often the error tolerance ϵ is violated. How fast N grows as ϵ and δ become smaller⁴ indicates how much data is needed to get good generalization.

We can use the VC bound to estimate the sample complexity for a given learning model. Fix $\delta > 0$, and suppose we want the generalization error to be at most ϵ . From Equation (2.12), the generalization error is bounded by $\sqrt{\frac{8}{N} \ln \frac{4m_{\mathcal{H}}(2N)}{\delta}}$, and so it suffices to make $\sqrt{\frac{8}{N} \ln \frac{4m_{\mathcal{H}}(2N)}{\delta}} \leq \epsilon$. It follows that

$$N \geq \frac{8}{\epsilon^2} \ln \left(\frac{4m_{\mathcal{H}}(2N)}{\delta} \right)$$

suffices to obtain generalization error at most ϵ (with probability at least $1 - \delta$). This gives an implicit bound for the sample complexity N , since N appears on both sides of the inequality. If we replace $m_{\mathcal{H}}(2N)$ in (2.12) by its polynomial upper bound in (2.10) which is based on the the VC dimension, we get a similar bound

$$N \geq \frac{8}{\epsilon^2} \ln \left(\frac{4((2N)^{d_{\text{vc}}} + 1)}{\delta} \right), \quad (2.13)$$

which is again implicit in N . We can obtain a numerical value for N using simple iterative methods.

Example 2.6. Suppose that we have a learning model with $d_{\text{vc}} = 3$ and would like the generalization error to be at most 0.1 with confidence 90% (so $\epsilon = 0.1$ and $\delta = 0.1$). How big a data set do we need? Using (2.13), we need

$$N \geq \frac{8}{0.1^2} \ln \left(\frac{4(2N)^3 + 4}{0.1} \right).$$

Trying an initial guess of $N = 1,000$ in the RHS, we get

$$N \geq \frac{8}{0.1^2} \ln \left(\frac{4(2 \times 1000)^3 + 4}{0.1} \right) \approx 21,193.$$

We then try the new value $N = 21,193$ in the RHS and continue this iterative process, rapidly converging to an estimate of $N \approx 30,000$. If d_{vc} were 4, a similar calculation will find that $N \approx 40,000$. For $d_{\text{vc}} = 5$, we get $N \approx 50,000$. You can see that the inequality suggests that the number of examples needed is approximately proportional to the VC dimension, as has been observed in practice. The constant of proportionality it suggests is 10,000, which is a *gross* overestimate; a more practical constant of proportionality is closer to 10. $\square$

⁴The term ‘complexity’ comes from a similar metaphor in computational complexity.

2.2.2 Penalty for Model Complexity

Sample complexity fixes the performance parameters ϵ (generalization error) and δ (confidence parameter) and estimates how many examples N are needed. In most practical situations, however, we are given a fixed data set $\mathcal{D}$, so N is also fixed. In this case, the relevant question is what performance can we expect given this particular N . The bound in (2.12) answers this question: with probability at least $1 - \delta$,

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{8}{N} \ln \left(\frac{4m_{\mathcal{H}}(2N)}{\delta} \right)}.$$

If we use the polynomial bound based on d_{vc} instead of $m_{\mathcal{H}}(2N)$, we get another valid bound on the out-of-sample error,

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{8}{N} \ln \left(\frac{4((2N)^{d_{\text{vc}}} + 1)}{\delta} \right)}. \quad (2.14)$$

Example 2.7. Suppose that $N = 100$ and we have a 90% confidence requirement ($\delta = 0.1$). We could ask what error bar can we offer with this confidence, if $\mathcal{H}$ has $d_{\text{vc}} = 1$. Using (2.14), we have

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \sqrt{\frac{8}{100} \ln \left(\frac{4(201)}{0.1} \right)} \approx E_{\text{in}}(g) + 0.848 \quad (2.15)$$

with confidence $\geq 90\%$. This is a pretty poor bound on E_{out} . Even if $E_{\text{in}} = 0$, E_{out} may still be close to 1. If $N = 1,000$, then we get $E_{\text{out}}(g) \leq E_{\text{in}}(g) + 0.301$, a somewhat more respectable bound. $\square$

Let us look more closely at the two parts that make up the bound on E_{out} in (2.12). The first part is E_{in} , and the second part is a term that increases as the VC dimension of $\mathcal{H}$ increases.

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \Omega(N, \mathcal{H}, \delta), \quad (2.16)$$

where

$$\begin{aligned} \Omega(N, \mathcal{H}, \delta) &= \sqrt{\frac{8}{N} \ln \left(\frac{4m_{\mathcal{H}}(2N)}{\delta} \right)} \\ &\leq \sqrt{\frac{8}{N} \ln \left(\frac{4((2N)^{d_{\text{vc}}} + 1)}{\delta} \right)}. \end{aligned}$$

One way to think of $\Omega(N, \mathcal{H}, \delta)$ is that it is a penalty for model complexity. It penalizes us by worsening the bound on E_{out} when we use a more complex $\mathcal{H}$ (larger d_{vc}). If someone manages to fit a simpler model with the same training

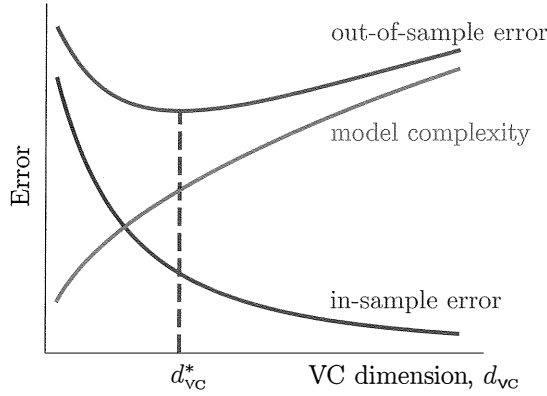


Figure 2.3: When we use a more complex learning model, one that has higher VC dimension d_{VC} , we are likely to fit the training data better resulting in a lower in sample error, but we pay a higher penalty for model complexity. A combination of the two, which estimates the out of sample error, thus attains a minimum at some intermediate d_{VC}^* .

error, they will get a more favorable estimate for E_{out} . The penalty $\Omega(N, \mathcal{H}, \delta)$ gets worse if we insist on higher confidence (lower δ), and it gets better when we have more training examples, as we would expect.

Although $\Omega(N, \mathcal{H}, \delta)$ goes up when $\mathcal{H}$ has a higher VC dimension, E_{in} is likely to go down with a higher VC dimension as we have more choices within $\mathcal{H}$ to fit the data. Therefore, we have a tradeoff: more complex models help E_{in} and hurt $\Omega(N, \mathcal{H}, \delta)$. The optimal model is a compromise that minimizes a combination of the two terms, as illustrated informally in Figure 2.3.

2.2.3 The Test Set

As we have seen, the generalization bound gives us a loose estimate of the out-of-sample error E_{out} based on E_{in} . While the estimate can be useful as a guideline for the training process, it is next to useless if the goal is to get an accurate forecast of E_{out} . If you are developing a system for a customer, you need a more accurate estimate so that your customer knows how well the system is expected to perform.

An alternative approach that we alluded to in the beginning of this chapter is to estimate E_{out} by using a *test set*, a data set that was not involved in the training process. The final hypothesis g is evaluated on the test set, and the result is taken as an estimate of E_{out} . We would like to now take a closer look at this approach.

Let us call the error we get on the test set E_{test} . When we report E_{test} as our estimate of E_{out} , we are in fact asserting that E_{test} generalizes very well to E_{out} . After all, E_{test} is just a sample estimate like E_{in} . How do we know

that E_{test} generalizes well? We can answer this question with authority now that we have developed the theory of generalization in concrete mathematical terms.

The effective number of hypotheses that matters in the generalization behavior of E_{test} is 1. There is only one hypothesis as far as the test set is concerned, and that's the final hypothesis g that the training phase produced. This hypothesis would not change if we used a different test set as it would if we used a different training set. Therefore, the simple Hoeffding Inequality is valid in the case of a test set. Had the choice of g been affected by the test set in any shape or form, it wouldn't be considered a *test* set any more and the simple Hoeffding Inequality would not apply.

Therefore, the generalization bound that applies to E_{test} is the simple Hoeffding Inequality with one hypothesis. This is a much tighter bound than the VC bound. For example, if you have 1,000 data points in the test set, E_{test} will be within $\pm 5\%$ of E_{out} with probability $\geq 98\%$. The bigger the test set you use, the more accurate E_{test} will be as an estimate of E_{out} .

Exercise 2.6

A data set has 600 examples. To properly test the performance of the final hypothesis, you set aside a randomly selected subset of 200 examples which are never used in the training phase; these form a test set. You use a learning model with 1,000 hypotheses and select the final hypothesis g based on the 400 training examples. We wish to estimate $E_{\text{out}}(g)$. We have access to two estimates: $E_{\text{in}}(g)$, the in sample error on the 400 training examples; and, $E_{\text{test}}(g)$, the test error on the 200 test examples that were set aside.

- (a) Using a 5% error tolerance ($\delta = 0.05$), which estimate has the higher 'error bar'?
- (b) Is there any reason why you shouldn't reserve even more examples for testing?

Another aspect that distinguishes the test set from the training set is that the test set is not biased. Both sets are finite samples that are bound to have some variance due to sample size, but the test set doesn't have an optimistic or pessimistic bias in its estimate of E_{out} . The training set has an optimistic bias, since it was used to choose a hypothesis that looked good *on it*. The VC generalization bound implicitly takes that bias into consideration, and that's why it gives a huge error bar. The test set just has straight finite-sample variance, but no bias. When you report the value of E_{test} to your customer and they try your system on new data, they are as likely to be pleasantly surprised as unpleasantly surprised, though quite likely not to be surprised at all.

There is a price to be paid for having a test set. The test set does not affect the outcome of our learning process, which only uses the training set. The test set just tells us how well we did. Therefore, if we set aside some

of the data points provided by the customer as a test set, we end up using fewer examples for training. Since the training set is used to select one of the hypotheses in $\mathcal{H}$, training examples are essential to finding a good hypothesis. If we take a big chunk of the data for testing and end up with too few examples for training, we may not get a good hypothesis from the training part even if we can reliably evaluate it in the testing part. We may end up reporting to the customer, with high confidence mind you, that the g we are delivering is terrible 😊. There is thus a tradeoff to setting aside test examples. We will address that tradeoff in more detail and learn some clever tricks to get around it in Chapter 4.

In some of the learning literature, E_{test} is used as synonymous with E_{out} . When we report experimental results in this book, we will often treat E_{test} based on a large test set as if it was E_{out} because of the closeness of the two quantities.

2.2.4 Other Target Types

Although the VC analysis was based on binary target functions, it can be extended to real-valued functions, as well as to other types of functions. The proofs in those cases are quite technical, and they do not add to the insight that the VC analysis of binary functions provides. Therefore, we will introduce an alternative approach that covers real-valued functions and provides new insights into generalization. The approach is based on bias-variance analysis, and will be discussed in the next section.

In order to deal with real-valued functions, we need to adapt the definitions of E_{in} and E_{out} that have so far been based on binary functions. We defined E_{in} and E_{out} in terms of binary error; either $h(\mathbf{x}) = f(\mathbf{x})$ or else $h(\mathbf{x}) \neq f(\mathbf{x})$. If f and h are real-valued, a more appropriate error measure would gauge how far $f(\mathbf{x})$ and $h(\mathbf{x})$ are from each other, rather than just whether their values are exactly the same.

An error measure that is commonly used in this case is the squared error $e(h(\mathbf{x}), f(\mathbf{x})) = (h(\mathbf{x}) - f(\mathbf{x}))^2$. We can define in-sample and out-of-sample versions of this error measure. The out-of-sample error is based on the expected value of the error measure over the entire input space $\mathcal{X}$,

$$E_{\text{out}}(h) = \mathbb{E} [(h(\mathbf{x}) - f(\mathbf{x}))^2],$$

while the in-sample error is based on averaging the error measure over the data set,

$$E_{\text{in}}(h) = \frac{1}{N} \sum_{n=1}^N (h(\mathbf{x}_n) - f(\mathbf{x}_n))^2.$$

These definitions make E_{in} a sample estimate of E_{out} just as it was in the case of binary functions. In fact, the error measure used for binary functions can also be expressed as a squared error.

Exercise 2.7

For binary target functions, show that $\mathbb{P}[h(\mathbf{x}) \neq f(\mathbf{x})]$ can be written as an expected value of a mean squared error measure in the following cases.

- (a) The convention used for the binary function is 0 or 1.
- (b) The convention used for the binary function is ± 1 .

[Hint: The difference between (a) and (b) is just a scale.]

Just as the sample frequency of error converges to the overall probability of error per Hoeffding's Inequality, the sample average of squared error converges to the expected value of that error (assuming finite variance). This is a manifestation of what is referred to as the 'law of large numbers' and Hoeffding's Inequality is just one form of that law. The same issues of the data set size and the hypothesis set complexity come into play just as they did in the VC analysis.

2.3 Approximation-Generalization Tradeoff

The VC analysis showed us that the choice of $\mathcal{H}$ needs to strike a balance between approximating f on the training data and generalizing on new data. The ideal $\mathcal{H}$ is a singleton hypothesis set containing only the target function. Unfortunately, we are better off buying a lottery ticket than hoping to have this $\mathcal{H}$. Since we do not know the target function, we resort to a larger model hoping that it will contain a good hypothesis, and hoping that the data will pin down that hypothesis. When you select your hypothesis set, you should balance these two conflicting goals; to have some hypothesis in $\mathcal{H}$ that can approximate f , and to enable the data to zoom in on the right hypothesis.

The VC generalization bound is one way to look at this tradeoff. If $\mathcal{H}$ is too simple, we may fail to approximate f well and end up with a large in-sample error term. If $\mathcal{H}$ is too complex, we may fail to generalize well because of the large model complexity term. There is another way to look at the approximation-generalization tradeoff which we will present in this section. It is particularly suited for squared error measures, rather than the binary error used in the VC analysis. The new way provides a different angle; instead of bounding E_{out} by E_{in} plus a penalty term Ω , we will decompose E_{out} into two different error terms.

2.3.1 Bias and Variance

The bias-variance decomposition of out-of-sample error is based on squared error measures. The out-of-sample error is

$$E_{\text{out}}(g^{(\mathcal{D})}) = \mathbb{E}_{\mathbf{x}} \left[(g^{(\mathcal{D})}(\mathbf{x}) - f(\mathbf{x}))^2 \right], \quad (2.17)$$

where $\mathbb{E}_{\mathbf{x}}$ denotes the expected value with respect to $\mathbf{x}$ (based on the probability distribution on the input space $\mathcal{X}$). We have made explicit the dependence of the final hypothesis g on the data $\mathcal{D}$, as this will play a key role in the current analysis. We can rid Equation (2.17) of the dependence on a particular data set by taking the expectation with respect to all data sets. We then get the expected out-of-sample error for our learning model, independent of any particular realization of the data set,

$$\begin{aligned}\mathbb{E}_{\mathcal{D}} [E_{\text{out}}(g^{(\mathcal{D})})] &= \mathbb{E}_{\mathcal{D}} [\mathbb{E}_{\mathbf{x}} [(g^{(\mathcal{D})}(\mathbf{x}) - f(\mathbf{x}))^2]] \\ &= \mathbb{E}_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}} [(g^{(\mathcal{D})}(\mathbf{x}) - f(\mathbf{x}))^2]] \\ &= \mathbb{E}_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}} [g^{(\mathcal{D})}(\mathbf{x})^2] - 2 \mathbb{E}_{\mathcal{D}} [g^{(\mathcal{D})}(\mathbf{x})] f(\mathbf{x}) + f(\mathbf{x})^2].\end{aligned}$$

The term $\mathbb{E}_{\mathcal{D}} [g^{(\mathcal{D})}(\mathbf{x})]$ gives an ‘average function’, which we denote by $\bar{g}(\mathbf{x})$. One can interpret $\bar{g}(\mathbf{x})$ in the following operational way. Generate many data sets $\mathcal{D}_1, \dots, \mathcal{D}_K$ and apply the learning algorithm to each data set to produce final hypotheses $g_1, \dots, g_K$. We can then estimate the average function for any $\mathbf{x}$ by $\bar{g}(\mathbf{x}) \approx \frac{1}{K} \sum_{k=1}^K g_k(\mathbf{x})$. Essentially, we are viewing $g(\mathbf{x})$ as a random variable, with the randomness coming from the randomness in the data set; $\bar{g}(\mathbf{x})$ is the expected value of this random variable (for a particular $\mathbf{x}$), and $\bar{g}$ is a *function*, the average function, composed of these expected values. The function $\bar{g}$ is a little counterintuitive; for one thing, $\bar{g}$ need not be in the model’s hypothesis set, even though it is the average of functions that are.

Exercise 2.8

- Show that if $\mathcal{H}$ is closed under linear combination (any linear combination of hypotheses in $\mathcal{H}$ is also a hypothesis in $\mathcal{H}$), then $\bar{g} \in \mathcal{H}$.
- Give a model for which the average function $\bar{g}$ is not in the model’s hypothesis set. [Hint: Use a very simple model.]
- For binary classification, do you expect $\bar{g}$ to be a binary function?

We can now rewrite the expected out-of-sample error in terms of $\bar{g}$:

$$\begin{aligned}\mathbb{E}_{\mathcal{D}} [E_{\text{out}}(g^{(\mathcal{D})})] &= \mathbb{E}_{\mathbf{x}} [\mathbb{E}_{\mathcal{D}} [g^{(\mathcal{D})}(\mathbf{x})^2] - 2\bar{g}(\mathbf{x})f(\mathbf{x}) + f(\mathbf{x})^2], \\ &= \mathbb{E}_{\mathbf{x}} \left[\underbrace{\mathbb{E}_{\mathcal{D}} [g^{(\mathcal{D})}(\mathbf{x})^2] - \bar{g}(\mathbf{x})^2}_{\mathbb{E}_{\mathcal{D}} [(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x}))^2]} + \underbrace{\bar{g}(\mathbf{x})^2 - 2\bar{g}(\mathbf{x})f(\mathbf{x}) + f(\mathbf{x})^2}_{(\bar{g}(\mathbf{x}) - f(\mathbf{x}))^2} \right],\end{aligned}$$

where the last reduction follows since $\bar{g}(\mathbf{x})$ is constant with respect to $\mathcal{D}$. The term $(\bar{g}(\mathbf{x}) - f(\mathbf{x}))^2$ measures how much the average function that we would learn using different data sets $\mathcal{D}$ deviates from the target function that generated these data sets. This term is appropriately called the bias:

$$\text{bias}(\mathbf{x}) = (\bar{g}(\mathbf{x}) - f(\mathbf{x}))^2,$$

as it measures how much our learning model is biased away from the target function.⁵ This is because $\bar{g}$ has the benefit of learning from an unlimited number of data sets, so it is only limited in its ability to approximate f by the limitation in the learning model itself. The term $\mathbb{E}_{\mathcal{D}} [(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x}))^2]$ is the variance of the random variable $g^{(\mathcal{D})}(\mathbf{x})$,

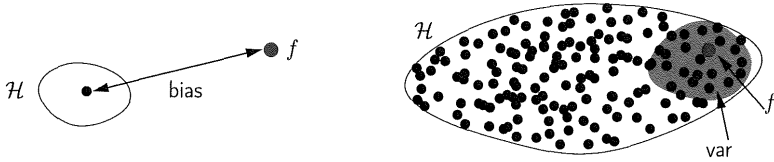
$$\text{var}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}} [(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x}))^2],$$

which measures the variation in the final hypothesis, depending on the data set. We thus arrive at the bias-variance decomposition of out-of-sample error,

$$\begin{aligned} \mathbb{E}_{\mathcal{D}} [E_{\text{out}}(g^{(\mathcal{D})})] &= \mathbb{E}_{\mathbf{x}} [\text{bias}(\mathbf{x}) + \text{var}(\mathbf{x})] \\ &= \text{bias} + \text{var}, \end{aligned}$$

where $\text{bias} = \mathbb{E}_{\mathbf{x}} [\text{bias}(\mathbf{x})]$ and $\text{var} = \mathbb{E}_{\mathbf{x}} [\text{var}(\mathbf{x})]$. Our derivation assumed that the data was noiseless. A similar derivation with noise in the data would lead to an additional noise term in the out-of-sample error (Problem 2.22). The noise term is unavoidable no matter what we do, so the terms we are interested in are really the bias and var.

The approximation-generalization tradeoff is captured in the bias-variance decomposition. To illustrate, let's consider two extreme cases: a very small model (with one hypothesis) and a very large one with all hypotheses.



Very small model. Since there is only one hypothesis, both the average function $\bar{g}$ and the final hypothesis $g^{(\mathcal{D})}$ will be the same, for any data set. Thus, $\text{var} = 0$. The bias will depend solely on how well this single hypothesis approximates the target f , and unless we are extremely lucky, we expect a large bias.

Very large model. The target function is in $\mathcal{H}$. Different data sets will lead to different hypotheses that agree with f on the data set, and are spread around f in the red region. Thus, $\text{bias} \approx 0$ because $\bar{g}$ is likely to be close to f . The var is large (heuristically represented by the size of the red region in the figure).

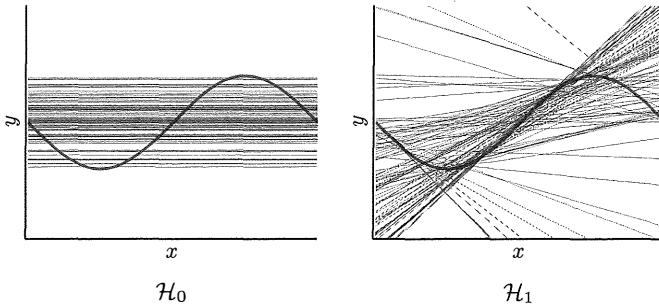
One can also view the variance as a measure of ‘instability’ in the learning model. Instability manifests in wild reactions to small variations or idiosyncrasies in the data, resulting in vastly different hypotheses.

⁵What we call bias is sometimes called bias² in the literature.

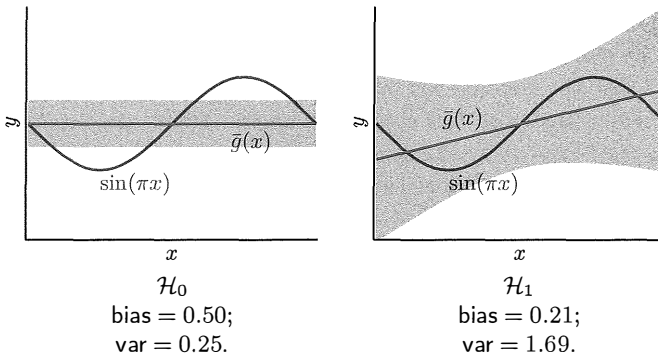
Example 2.8. Consider a target function $f(x) = \sin(\pi x)$ and a data set of size $N = 2$. We sample x uniformly in $[-1, 1]$ to generate a data set $(x_1, y_1), (x_2, y_2)$; and fit the data using one of two models:

- $\mathcal{H}_0$: Set of all lines of the form $h(x) = b$;
 $\mathcal{H}_1$: Set of all lines of the form $h(x) = ax + b$.

For $\mathcal{H}_0$, we choose the constant hypothesis that best fits the data (the horizontal line at the midpoint, $b = \frac{y_1 + y_2}{2}$). For $\mathcal{H}_1$, we choose the line that passes through the two data points (x_1, y_1) and (x_2, y_2) . Repeating this process with many data sets, we can estimate the bias and the variance. The figures which follow show the resulting fits on the same (random) data sets for both models.



With $\mathcal{H}_1$, the learned hypothesis is wilder and varies extensively depending on the data set. The bias-var analysis is summarized in the next figures.



Average hypothesis $\bar{g}$ (red) with $\text{var}(x)$ indicated by the gray shaded region that is $\bar{g}(x) \pm \sqrt{\text{var}(x)}$.

For $\mathcal{H}_1$, the average hypothesis $\bar{g}$ (red line) is a reasonable fit with a fairly small bias of 0.21. However, the large variability leads to a high var of 1.69 resulting in a large expected out-of-sample error of 1.90. With the simpler

model $\mathcal{H}_0$, the fits are much less volatile and we have a significantly lower **var** of 0.25, as indicated by the shaded region. However, the average fit is now the zero function, resulting in a higher **bias** of 0.50. The total out-of-sample error has a much smaller expected value of 0.75. The simpler model wins by significantly decreasing the **var** at the expense of a smaller increase in **bias**.

Notice that we are not comparing how well the red curves (the average hypotheses) fit the sine. These curves are only conceptual, since in real learning we do not have access to the multitude of data sets needed to generate them. We have one data set, and the simpler model results in a better out-of-sample error on average as we fit our model to just this one data. However, the **var** term decreases as N increases, so if we get a bigger and bigger data set, the **bias** term will be the dominant part of E_{out} , and $\mathcal{H}_1$ will win. $\square$

The learning algorithm plays a role in the bias-variance analysis that it did not play in the VC analysis. Two points are worth noting.

1. By design, the VC analysis is based purely on the hypothesis set $\mathcal{H}$, independently of the learning algorithm $\mathcal{A}$. In the bias-variance analysis, both $\mathcal{H}$ and the algorithm $\mathcal{A}$ matter. With the same $\mathcal{H}$, using a different learning algorithm can produce a different $g^{(\mathcal{D})}$. Since $g^{(\mathcal{D})}$ is the building block of the bias-variance analysis, this may result in different **bias** and **var** terms.
2. Although the bias-variance analysis is based on squared-error measure, the learning algorithm itself does not have to be based on minimizing the squared error. It can use any criterion to produce $g^{(\mathcal{D})}$ based on $\mathcal{D}$. However, once the algorithm produces $g^{(\mathcal{D})}$, we measure its **bias** and **variance** using squared error.

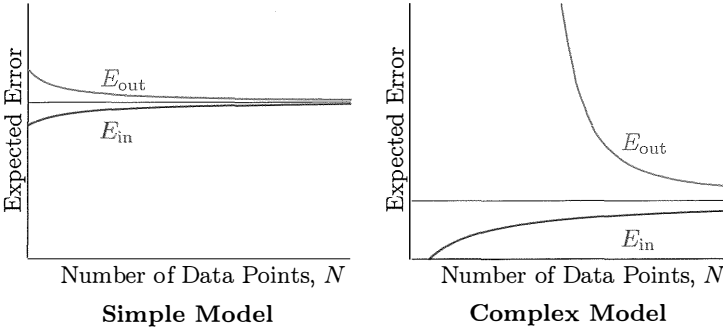
Unfortunately, the **bias** and **variance** cannot be computed in practice, since they depend on the target function and the input probability distribution (both unknown). Thus, the bias-variance decomposition is a conceptual tool which is helpful when it comes to developing a model. There are two typical goals when we consider **bias** and **variance**. The first is to try to lower the **variance** without significantly increasing the **bias**, and the second is to lower the **bias** without significantly increasing the **variance**. These goals are achieved by different techniques, some principled and some heuristic. Regularization is one of these techniques that we will discuss in Chapter 4. Reducing the **bias** without increasing the **variance** requires some prior information regarding the target function to steer the selection of $\mathcal{H}$ in the direction of f , and this task is largely application-specific. On the other hand, reducing the **variance** without compromising the **bias** can be done through general techniques.

2.3.2 The Learning Curve

We close this chapter with an important plot that illustrates the tradeoffs that we have seen so far. The *learning curves* summarize the behavior of the

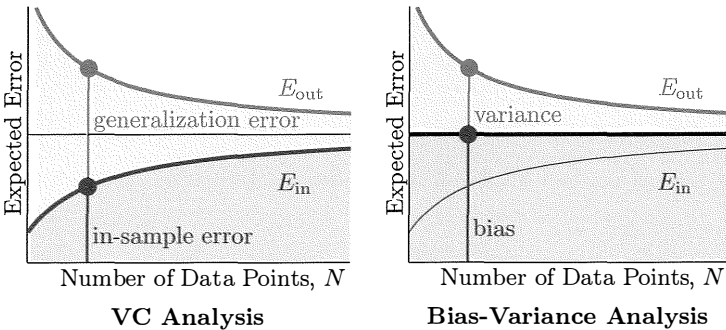
in-sample and out-of-sample errors as we vary the size of the training set.

After learning with a particular data set $\mathcal{D}$ of size N , the final hypothesis $g^{(\mathcal{D})}$ has in-sample error $E_{\text{in}}(g^{(\mathcal{D})})$ and out-of-sample error $E_{\text{out}}(g^{(\mathcal{D})})$, both of which depend on $\mathcal{D}$. As we saw in the bias-variance analysis, the expectation with respect to all data sets of size N gives the expected errors: $\mathbb{E}_{\mathcal{D}}[E_{\text{in}}(g^{(\mathcal{D})})]$ and $\mathbb{E}_{\mathcal{D}}[E_{\text{out}}(g^{(\mathcal{D})})]$. These expected errors are functions of N , and are called the learning curves of the model. We illustrate the learning curves for a simple learning model and a complex one, based on actual experiments.



Notice that for the simple model, the learning curves converge more quickly but to worse ultimate performance than for the complex model. This behavior is typical in practice. For both simple and complex models, the out-of-sample learning curve is decreasing in N , while the in-sample learning curve is increasing in N . Let us take a closer look at these curves and interpret them in terms of the different approaches to generalization that we have discussed.

In the VC analysis, E_{out} was expressed as the sum of E_{in} and a generalization error that was bounded by Ω , the penalty for model complexity. In the bias-variance analysis, E_{out} was expressed as the sum of a bias and a variance. The following learning curves illustrate these two approaches side by side.



The VC analysis bounds the generalization error which is illustrated on the left.⁶ The bias-variance analysis is illustrated on the right. The bias-variance illustration is somewhat idealized, since it assumes that, for every N , the average learned hypothesis $\bar{g}$ has the same performance as the best approximation to f in the learning model.

When the number of data points increases, we move to the right on the learning curves and both the generalization error and the variance term shrink, as expected. The learning curve also illustrates an important point about E_{in} . As N increases, E_{in} edges toward the smallest error that the learning model can achieve in approximating f . For small N , the value of E_{in} is actually smaller than that ‘smallest possible’ error. This is because the learning model has an easier task for smaller N ; it only needs to approximate f on the N points regardless of what happens outside those points. Therefore, it can achieve a superior fit on those points, albeit at the expense of an inferior fit on the rest of the points as shown by the corresponding value of E_{out} .

⁶For the learning curve, we take the expected values of all quantities with respect to $\mathcal{D}$ of size N .

2.4 Problems

Problem 2.1 In Equation (2.1), set $\delta = 0.03$ and let

$$\epsilon(M, N, \delta) = \sqrt{\frac{1}{2N} \ln \frac{2M}{\delta}}.$$

- (a) For $M = 1$, how many examples do we need to make $\epsilon \leq 0.05$?
- (b) For $M = 100$, how many examples do we need to make $\epsilon \leq 0.05$?
- (c) For $M = 10,000$, how many examples do we need to make $\epsilon \leq 0.05$?

Problem 2.2 Show that for the learning model of positive rectangles (aligned horizontally or vertically), $m_{\mathcal{H}}(4) = 2^4$ and $m_{\mathcal{H}}(5) < 2^5$. Hence, give a bound for $m_{\mathcal{H}}(N)$.

Problem 2.3 Compute the maximum number of dichotomies, $m_{\mathcal{H}}(N)$, for these learning models, and consequently compute d_{VC} , the VC dimension.

- (a) Positive or negative ray: $\mathcal{H}$ contains the functions which are +1 on $[a, \infty)$ (for some a) together with those that are +1 on $(-\infty, a]$ (for some a).
- (b) Positive or negative interval: $\mathcal{H}$ contains the functions which are +1 on an interval $[a, b]$ and -1 elsewhere or -1 on an interval $[a, b]$ and +1 elsewhere.
- (c) Two concentric spheres in $\mathbb{R}^d$: $\mathcal{H}$ contains the functions which are +1 for $a \leq \sqrt{x_1^2 + \dots + x_d^2} \leq b$.

Problem 2.4 Show that $B(N, k) = \sum_{i=0}^{k-1} \binom{N}{i}$ by showing the other direction to Lemma 2.3, namely that

$$B(N, k) \geq \sum_{i=0}^{k-1} \binom{N}{i}.$$

To do so, construct a specific set of $\sum_{i=0}^{k-1} \binom{N}{i}$ dichotomies that does not shatter any subset of k variables. [Hint: Try limiting the number of -1's in each dichotomy.]

Problem 2.5 Prove by induction that $\sum_{i=0}^D \binom{N}{i} \leq N^D + 1$, hence

$$m_{\mathcal{H}}(N) \leq N^{d_{\text{VC}}} + 1.$$

Problem 2.6 Prove that for $N \geq d$,

$$\sum_{i=0}^d \binom{N}{i} \leq \left(\frac{eN}{d} \right)^d.$$

We suggest you first show the following intermediate steps.

- (a) $\sum_{i=0}^d \binom{N}{i} \leq \sum_{i=0}^d \binom{N}{i} \left(\frac{N}{d} \right)^d \leq \left(\frac{N}{d} \right)^d \sum_{i=0}^N \binom{N}{i} \left(\frac{d}{N} \right)^i.$
- (b) $\sum_{i=0}^N \binom{N}{i} \left(\frac{d}{N} \right)^i \leq e^d. [Hints: Binomial theorem; $(1 + \frac{1}{x})^x \leq e$ for $x > 0$.]$

Hence, argue that $m_{\mathcal{H}}(N) \leq \left(\frac{eN}{d_{\text{vc}}} \right)^{d_{\text{vc}}}.$

Problem 2.7 Plot the bounds for $m_{\mathcal{H}}(N)$ given in Problems 2.5 and 2.6 for $d_{\text{vc}} = 2$ and $d_{\text{vc}} = 5$. When do you prefer one bound over the other?

Problem 2.8 Which of the following are possible growth functions $m_{\mathcal{H}}(N)$ for some hypothesis set:

$$1+N; \quad 1+N+\frac{N(N-1)}{2}; \quad 2^N; \quad 2^{\lfloor \sqrt{N} \rfloor}; \quad 2^{\lfloor N/2 \rfloor}; \quad 1+N+\frac{N(N-1)(N-2)}{6}.$$

Problem 2.9 [hard] For the perceptron in d dimensions, show that

$$m_{\mathcal{H}}(N) = 2 \sum_{i=0}^d \binom{N-1}{i}. \quad [Hint: Cover(1965) in Further Reading.]$$

Use this formula to verify that $d_{\text{vc}} = d + 1$ by evaluating $m_{\mathcal{H}}(d + 1)$ and $m_{\mathcal{H}}(d + 2)$. Plot $m_{\mathcal{H}}(N)/2^N$ for $d = 10$ and $N \in [1, 40]$. If you generate a random dichotomy on N points in 10 dimensions, give an upper bound on the probability that the dichotomy will be separable for $N = 10, 20, 40$.

Problem 2.10 Show that $m_{\mathcal{H}}(2N) \leq m_{\mathcal{H}}(N)^2$, and hence obtain a generalization bound which only involves $m_{\mathcal{H}}(N)$.

Problem 2.11 Suppose $m_{\mathcal{H}}(N) = N + 1$, so $d_{\text{vc}} = 1$. You have 100 training examples. Use the generalization bound to give a bound for E_{out} with confidence 90%. Repeat for $N = 10,000$.

Problem 2.12 For an $\mathcal{H}$ with $d_{\text{vc}} = 10$, what sample size do you need (as prescribed by the generalization bound) to have a 95% confidence that your generalization error is at most 0.05?

Problem 2.13

- (a) Let $\mathcal{H} = \{h_1, h_2, \dots, h_M\}$ with some finite M . Prove that $d_{\text{vc}}(\mathcal{H}) \leq \log_2 M$.
- (b) For hypothesis sets $\mathcal{H}_1, \mathcal{H}_2, \dots, \mathcal{H}_K$ with finite VC dimensions $d_{\text{vc}}(\mathcal{H}_k)$, derive and prove the tightest upper and lower bound that you can get on $d_{\text{vc}}(\cap_{k=1}^K \mathcal{H}_k)$.
- (c) For hypothesis sets $\mathcal{H}_1, \mathcal{H}_2, \dots, \mathcal{H}_K$ with finite VC dimensions $d_{\text{vc}}(\mathcal{H}_k)$, derive and prove the tightest upper and lower bounds that you can get on $d_{\text{vc}}(\cup_{k=1}^K \mathcal{H}_k)$.

Problem 2.14 Let $\mathcal{H}_1, \mathcal{H}_2, \dots, \mathcal{H}_K$ be K hypothesis sets with finite VC dimension d_{vc} . Let $\mathcal{H} = \mathcal{H}_1 \cup \mathcal{H}_2 \cup \dots \cup \mathcal{H}_K$ be the union of these models.

- (a) Show that $d_{\text{vc}}(\mathcal{H}) < K(d_{\text{vc}} + 1)$.
- (b) Suppose that ℓ satisfies $2^\ell > 2K\ell^{d_{\text{vc}}}$. Show that $d_{\text{vc}}(\mathcal{H}) \leq \ell$.
- (c) Hence, show that

$$d_{\text{vc}}(\mathcal{H}) \leq \min(K(d_{\text{vc}} + 1), 7(d_{\text{vc}} + K) \log_2(d_{\text{vc}} K)).$$

That is, $d_{\text{vc}}(\mathcal{H}) = O(\max(d_{\text{vc}}, K) \log_2 \max(d_{\text{vc}}, K))$ is not too bad.

Problem 2.15 The monotonically increasing hypothesis set is

$$\mathcal{H} = \{h \mid \mathbf{x}_1 \geq \mathbf{x}_2 \implies h(\mathbf{x}_1) \geq h(\mathbf{x}_2)\},$$

where $\mathbf{x}_1 \geq \mathbf{x}_2$ if and only if the inequality is satisfied for every component.

- (a) Give an example of a monotonic classifier in two dimensions, clearly showing the +1 and -1 regions.
- (b) Compute $m_{\mathcal{H}}(N)$ and hence the VC dimension. [Hint: Consider a set of N points generated by first choosing one point, and then generating the next point by increasing the first component and decreasing the second component until N points are obtained.]

Problem 2.16 In this problem, we will consider $\mathcal{X} = \mathbb{R}$. That is, x is a one dimensional variable. For a hypothesis set

$$\mathcal{H} = \left\{ h_c \mid h_c(x) = \text{sign} \left(\sum_{i=0}^D c_i x^i \right) \right\},$$

prove that the VC dimension of $\mathcal{H}$ is exactly $(D + 1)$ by showing that

- (a) There are $(D + 1)$ points which are shattered by $\mathcal{H}$.
- (b) There are no $(D + 2)$ points which are shattered by $\mathcal{H}$.

Problem 2.17 The VC dimension depends on the input space as well as $\mathcal{H}$. For a fixed $\mathcal{H}$, consider two input spaces $\mathcal{X}_1 \subseteq \mathcal{X}_2$. Show that the VC dimension of $\mathcal{H}$ with respect to input space $\mathcal{X}_1$ is at most the VC dimension of $\mathcal{H}$ with respect to input space $\mathcal{X}_2$.

How can the result of this problem be used to answer part (b) in Problem 2.16? [Hint: How is Problem 2.16 related to a perceptron in D dimensions?]

Problem 2.18 The VC dimension of the perceptron hypothesis set corresponds to the number of parameters $(w_0, w_1, \dots, w_d)$ of the set, and this observation is 'usually' true for other hypothesis sets. However, we will present a counter example here. Prove that the following hypothesis set for $x \in \mathbb{R}$ has an infinite VC dimension:

$$\mathcal{H} = \left\{ h_\alpha \mid h_\alpha(x) = (-1)^{\lfloor \alpha x \rfloor}, \text{ where } \alpha \in \mathbb{R} \right\},$$

where $\lfloor A \rfloor$ is the biggest integer $\leq A$ (the floor function). This hypothesis has only one parameter α but 'enjoys' an infinite VC dimension. [Hint: Consider $x_1, \dots, x_N$, where $x_n = 10^n$, and show how to implement an arbitrary dichotomy $y_1, \dots, y_N$.]

Problem 2.19 This problem derives a bound for the VC dimension of a complex hypothesis set that is built from simpler hypothesis sets via composition. Let $\mathcal{H}_1, \dots, \mathcal{H}_K$ be hypothesis sets with VC dimension $d_1, \dots, d_K$. Fix $h_1, \dots, h_K$, where $h_i \in \mathcal{H}_i$. Define a vector $\mathbf{z}$ obtained from $\mathbf{x}$ to have components $h_i(\mathbf{x})$. Note that $\mathbf{x} \in \mathbb{R}^d$, but $\mathbf{z} \in \{-1, +1\}^K$. Let $\tilde{\mathcal{H}}$ be a hypothesis set of functions that take inputs in $\mathbb{R}^K$. So

$$\tilde{h} \in \tilde{\mathcal{H}}: \mathbf{z} \in \mathbb{R}^K \mapsto \{+1, -1\},$$

and suppose that $\tilde{\mathcal{H}}$ has VC dimension $\tilde{d}$.

We can apply a hypothesis in $\tilde{\mathcal{H}}$ to the $\mathbf{z}$ constructed from $(h_1, \dots, h_K)$. This is the composition of the hypothesis set $\tilde{\mathcal{H}}$ with $(\mathcal{H}_1, \dots, \mathcal{H}_K)$. More formally, the composed hypothesis set $\mathcal{H} = \tilde{\mathcal{H}} \circ (\mathcal{H}_1, \dots, \mathcal{H}_K)$ is defined by $h \in \mathcal{H}$ if

$$h(\mathbf{x}) = \tilde{h}(h_1(\mathbf{x}), \dots, h_K(\mathbf{x})), \quad \tilde{h} \in \tilde{\mathcal{H}}; \quad h_i \in \mathcal{H}_i.$$

(a) Show that

$$m_{\mathcal{H}}(N) \leq m_{\tilde{\mathcal{H}}}(N) \prod_{i=1}^K m_{\mathcal{H}_i}(N). \quad (2.18)$$

[Hint: Fix N points $\mathbf{x}_1, \dots, \mathbf{x}_N$ and fix $h_1, \dots, h_K$. This generates N transformed points $\mathbf{z}_1, \dots, \mathbf{z}_N$. These $\mathbf{z}_1, \dots, \mathbf{z}_N$ can be dichotomized in at most $m_{\tilde{\mathcal{H}}}(N)$ ways, hence for fixed $(h_1, \dots, h_K)$, $(\mathbf{x}_1, \dots, \mathbf{x}_N)$ can be dichotomized in at most $m_{\tilde{\mathcal{H}}}(N)$ ways. Through the eyes of $\mathbf{x}_1, \dots, \mathbf{x}_N$, at most how many hypotheses are there (effectively) in $\mathcal{H}_i$? Use this bound to bound the effective number of K -tuples $(h_1, \dots, h_K)$ that need to be considered. Finally, argue that you can bound the number of dichotomies that can be implemented by the product of the number of possible K -tuples $(h_1, \dots, h_K)$ and the number of dichotomies per K -tuple.]

(b) Use the bound $m(N) \leq \left(\frac{eN}{d_{\text{vc}}}\right)^{d_{\text{vc}}}$ to get a bound for $m_{\mathcal{H}}(N)$ in terms of $\tilde{d}, d_1, \dots, d_K$.

(c) Let $D = \tilde{d} + \sum_{i=1}^K d_i$, and assume that $D > 2e \log_2 D$. Show that

$$d_{\text{vc}}(\mathcal{H}) \leq 2D \log_2 D.$$

(d) If $\mathcal{H}_i$ and $\tilde{\mathcal{H}}$ are all perceptron hypothesis sets, show that

$$d_{\text{vc}}(\mathcal{H}) = O(dK \log(dK)).$$

In the next chapter, we will further develop the simple linear model. This linear model is the building block of many other models, such as neural networks. The results of this problem show how to bound the VC dimension of the more complex models built in this manner.

Problem 2.20 There are a number of bounds on the generalization error ϵ , all holding with probability at least $1 - \delta$.

(a) Original VC-bound:

$$\epsilon \leq \sqrt{\frac{8}{N} \ln \frac{4m_{\mathcal{H}}(2N)}{\delta}}.$$

(b) Rademacher Penalty Bound:

$$\epsilon \leq \sqrt{\frac{2 \ln(2N m_{\mathcal{H}}(N))}{N}} + \sqrt{\frac{2}{N} \ln \frac{1}{\delta}} + \frac{1}{N}.$$

(continued on next page)

(c) Parrondo and Van den Broek:

$$\epsilon \leq \sqrt{\frac{1}{N} \left(2\epsilon + \ln \frac{6m_{\mathcal{H}}(2N)}{\delta} \right)}.$$

(d) Devroye:

$$\epsilon \leq \sqrt{\frac{1}{2N} \left(4\epsilon(1 + \epsilon) + \ln \frac{4m_{\mathcal{H}}(N^2)}{\delta} \right)}.$$

Note that (c) and (d) are implicit bounds in ϵ . Fix $d_{\text{VC}} = 50$ and $\delta = 0.05$ and plot these bounds as a function of N . Which is best?

Problem 2.21 Assume the following theorem to hold

Theorem

$$\mathbb{P} \left[\frac{E_{\text{out}}(\mathbf{g}) - E_{\text{in}}(\mathbf{g})}{\sqrt{E_{\text{out}}(\mathbf{g})}} > \epsilon \right] \leq c \cdot m_{\mathcal{H}}(2N) \exp \left(-\frac{\epsilon^2 N}{4} \right),$$

where c is a constant that is a little bigger than 6.

This bound is useful because sometimes what we care about is not the absolute generalization error but instead a relative generalization error (one can imagine that a generalization error of 0.01 is more significant when $E_{\text{out}} = 0.01$ than when $E_{\text{out}} = 0.5$). Convert this to a generalization bound by showing that with probability at least $1 - \delta$,

$$E_{\text{out}}(g) \leq E_{\text{in}}(g) + \frac{\xi}{2} \left[1 + \sqrt{1 + \frac{4E_{\text{in}}(g)}{\xi}} \right],$$

where $\xi = \frac{4}{N} \log \frac{c \cdot m_{\mathcal{H}}(2N)}{\delta}$.

Problem 2.22 When there is noise in the data, $E_{\text{out}}(g^{(\mathcal{D})}) = \mathbb{E}_{\mathbf{x}, y}[(g^{(\mathcal{D})}(\mathbf{x}) - y(\mathbf{x}))^2]$, where $y(\mathbf{x}) = f(\mathbf{x}) + \epsilon$. If ϵ is a zero mean noise random variable with variance σ^2 , show that the bias variance decomposition becomes

$$\mathbb{E}_{\mathcal{D}}[E_{\text{out}}(g^{(\mathcal{D})})] = \sigma^2 + \text{bias} + \text{var}.$$

Problem 2.23 Consider the learning problem in Example 2.8, where the input space is $\mathcal{X} = [-1, +1]$, the target function is $f(x) = \sin(\pi x)$, and the input probability distribution is uniform on $\mathcal{X}$. Assume that the training set $\mathcal{D}$ has only two data points (picked independently), and that the learning algorithm picks the hypothesis that minimizes the in sample mean squared error. In this problem, we will dig deeper into this case.

For each of the following learning models, find (analytically or numerically) (i) the best hypothesis that approximates f in the mean squared error sense (assume that f is known for this part), (ii) the expected value (with respect to $\mathcal{D}$) of the hypothesis that the learning algorithm produces, and (iii) the expected out of sample error and its bias and var components.

- (a) The learning model consists of all hypotheses of the form $h(x) = ax + b$ (if you need to deal with the infinitesimal probability case of two identical data points, choose the hypothesis tangential to f).
- (b) The learning model consists of all hypotheses of the form $h(x) = ax$. This case was not covered in Example 2.8.
- (c) The learning model consists of all hypotheses of the form $h(x) = b$.

Problem 2.24 Consider a simplified learning scenario. Assume that the input dimension is one. Assume that the input variable x is uniformly distributed in the interval $[-1, 1]$. The data set consists of 2 points $\{x_1, x_2\}$ and assume that the target function is $f(x) = x^2$. Thus, the full data set is $\mathcal{D} = \{(x_1, x_1^2), (x_2, x_2^2)\}$. The learning algorithm returns the line fitting these two points as g ($\mathcal{H}$ consists of functions of the form $h(x) = ax + b$). We are interested in the test performance (E_{out}) of our learning system with respect to the squared error measure, the bias and the var.

- (a) Give the analytic expression for the average function $\bar{g}(x)$.
- (b) Describe an experiment that you could run to determine (numerically) $\bar{g}(x)$, E_{out} , bias, and var.
- (c) Run your experiment and report the results. Compare E_{out} with bias+var. Provide a plot of your $\bar{g}(x)$ and $f(x)$ (on the same plot).
- (d) Compute analytically what E_{out} , bias and var should be.
